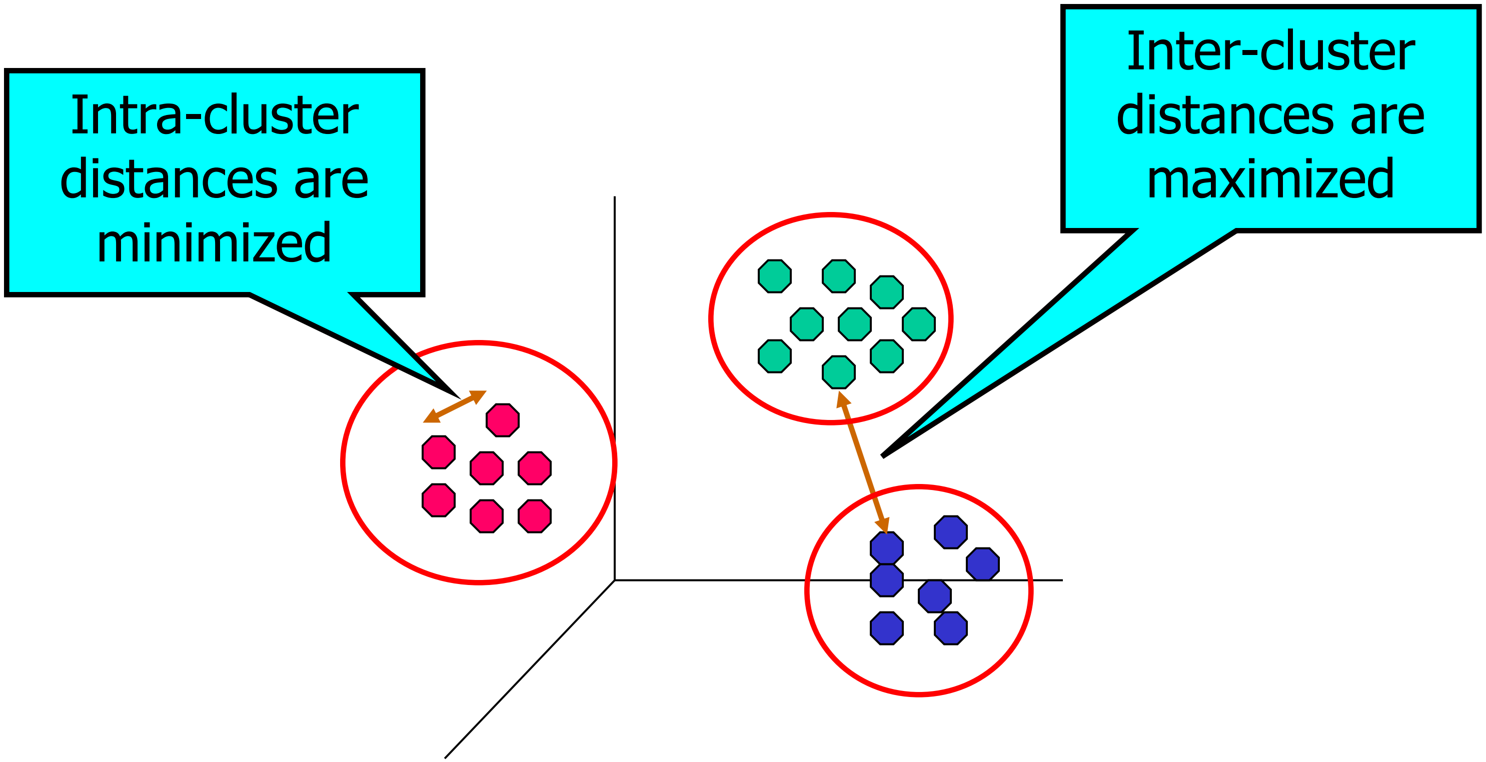
Cluster Analysis

What is Cluster Analysis?

Finding groups of objects such that the objects in a group will be **similar** (or related) **to** one another and **different from** (or unrelated to) the objects in other groups



Applications of Cluster Analysis

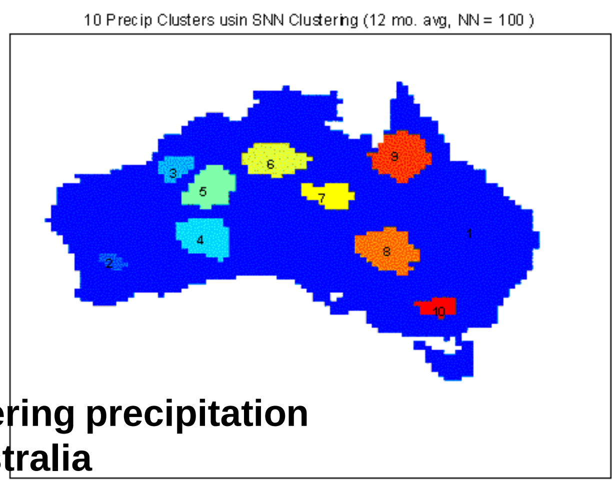
Understanding

Group related documents for browsing, group genes and proteins that have similar functionality, or group stocks with similar price fluctuations

	Discovered Clusters	*Industry Group*
1	Applied-Matl-DOWN,Bay-Network-Down,3-COM-DOWN, Cabletron-Sys-DOWN,CISCO-DOWN,HP-DOWN, DSC-Comm-DOWN,INTEL-DOWN,LSI-Logic-DOWN, Micron-Tech-DOWN,Texas-Inst-Down,Tellabs-Inc-Down, Natl-Semiconduct-DOWN,Oracl-DOWN,SGI-DOWN, Sun-DOWN	Technology1-DOWN
2	Apple-Comp-DOWN,Autodesk-DOWN,DEC-DOWN, ADV-Micro-Device-DOWN,Andrew-Corp-DOWN, Computer-Assoc-DOWN,Circuit-City-DOWN, Compaq-DOWN, EMC-Corp-DOWN, Gen-Inst-DOWN, Motorola-DOWN,Microsoft-DOWN,Scientific-Atl-DOWN	Technology2-DOWN
3	Fannie-Mae-DOWN,Fed-Home-Loan-DOWN, MBNA-Corp-DOWN,Morgan-Stanley-DOWN	Financial-DOWN
4	Baker-Hughes-UP,Dresser-Inds-UP,Halliburton-HLD-UP, Louisiana-Land-UP,Phillips-Petro-UP,Unocal-UP, Schlumberger-UP	Oil-UP

Summarization

Reduce the size of large data sets



Clustering precipitation in Australia

What is not Cluster Analysis?

- Supervised classification

  Have class label information

- Simple segmentation

  Dividing students into different registration groups alphabetically, by last name

- Results of a query

  Groupings are a result of an external specification

- Graph partitioning

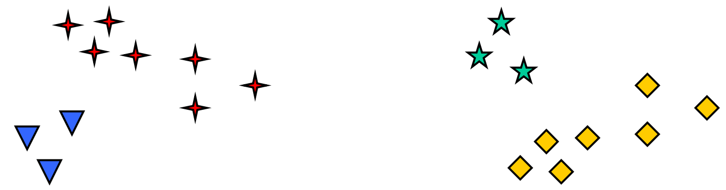
  Some mutual relevance and synergy, but areas are not identical

Notion of a Cluster can be Ambiguous

How many clusters?

Six Clusters

Two Clusters



Four Clusters

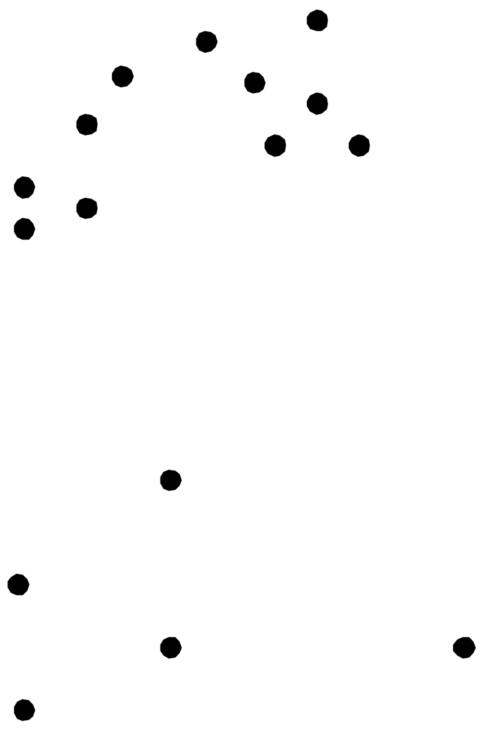
Types of Clusterings

- A clustering is a set of clusters
- Important distinction between hierarchical and partitional sets of clusters
- Partitional Clustering

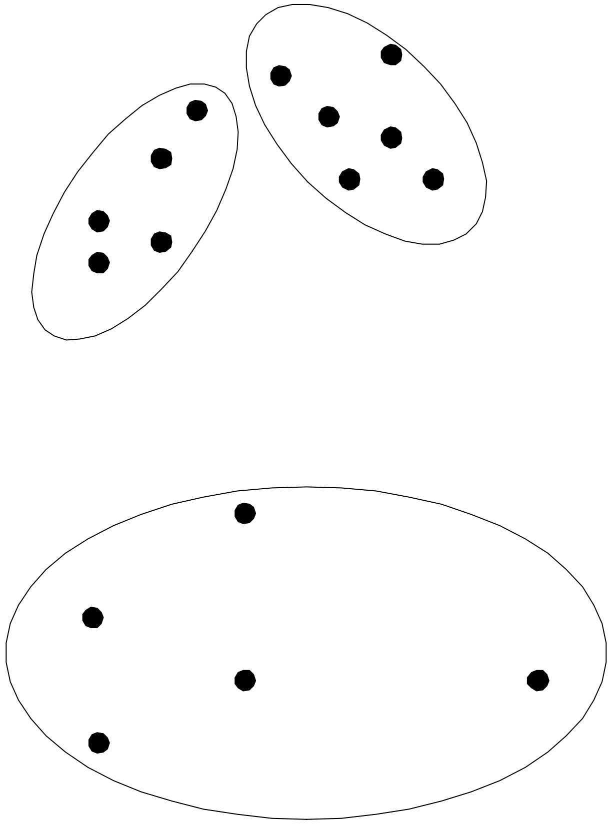
  A division data objects into non-overlapping subsets (clusters) such that each data object is in exactly one subset
- Hierarchical clustering

  A set of nested clusters organized as a hierarchical tree

Partitional Clustering

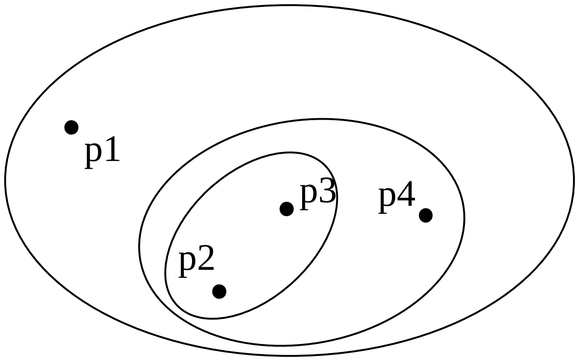


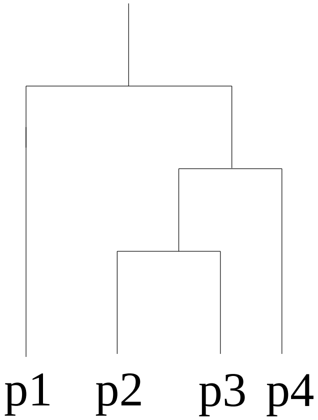
Original Points



A Partitional Clustering

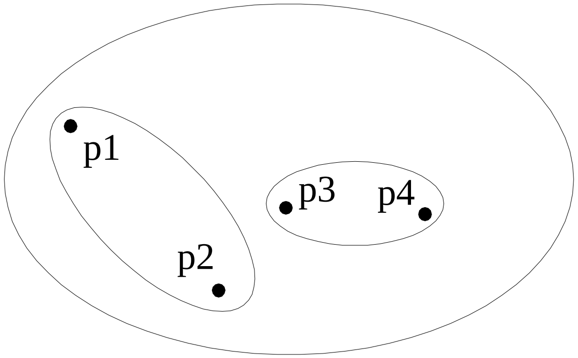
Hierarchical Clustering

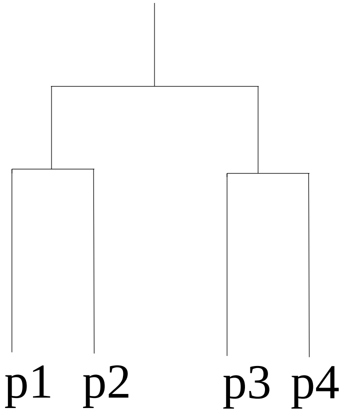




Traditional Hierarchical Clustering

Traditional Dendrogram





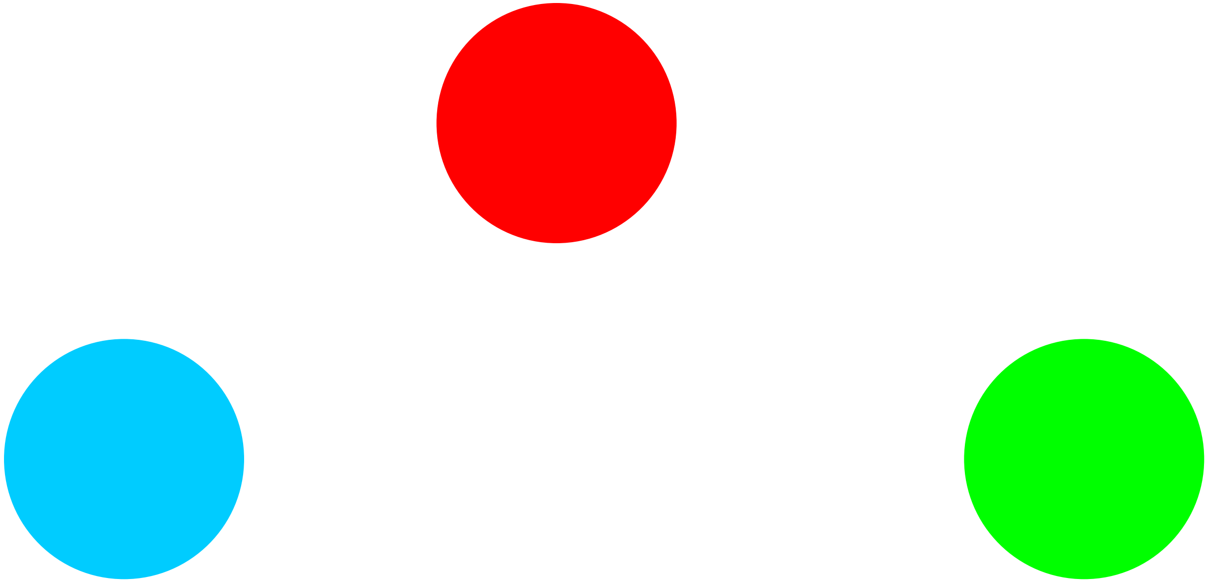
Non-traditional Hierarchical Clustering

Non-traditional Dendrogram

Types of Clusters: Well-Separated

Well-Separated Clusters:

A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.



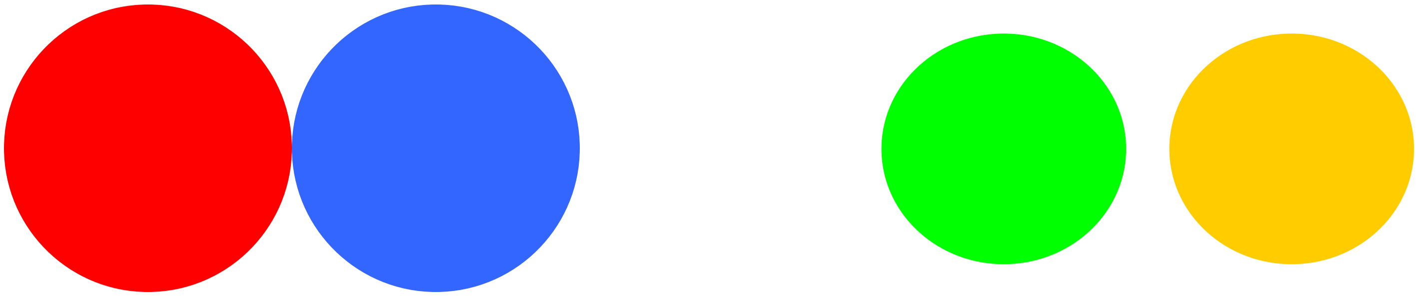
3 well-separated clusters

Types of Clusters: Center-Based

Center-based

A cluster is a set of objects such that an object in a cluster is closer (more similar) to the "center" of a cluster, than to the center of any other cluster

The center of a cluster is often a centroid, the average of all the points in the cluster, or a medoid, the most "representative" point of a cluster

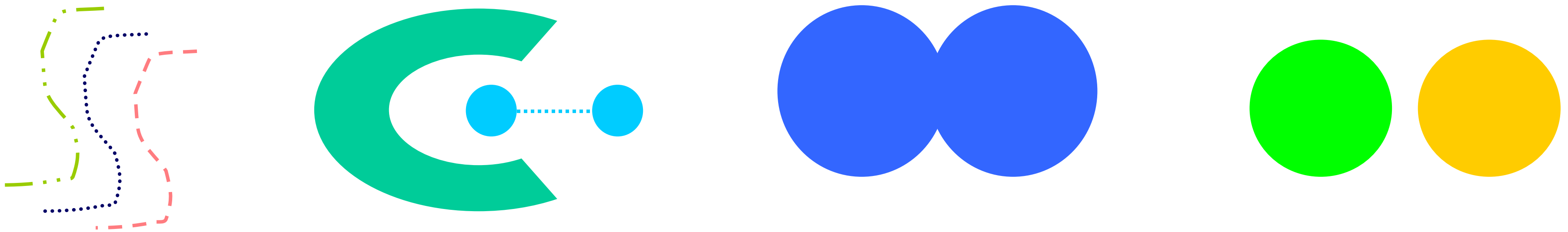


4 center-based clusters

Types of Clusters: Contiguity-Based

Contiguous Cluster (Nearest neighbor or Transitive)

A cluster is a set of points such that a point in a cluster is closer (or more similar) to one or more other points in the cluster than to any point not in the cluster.



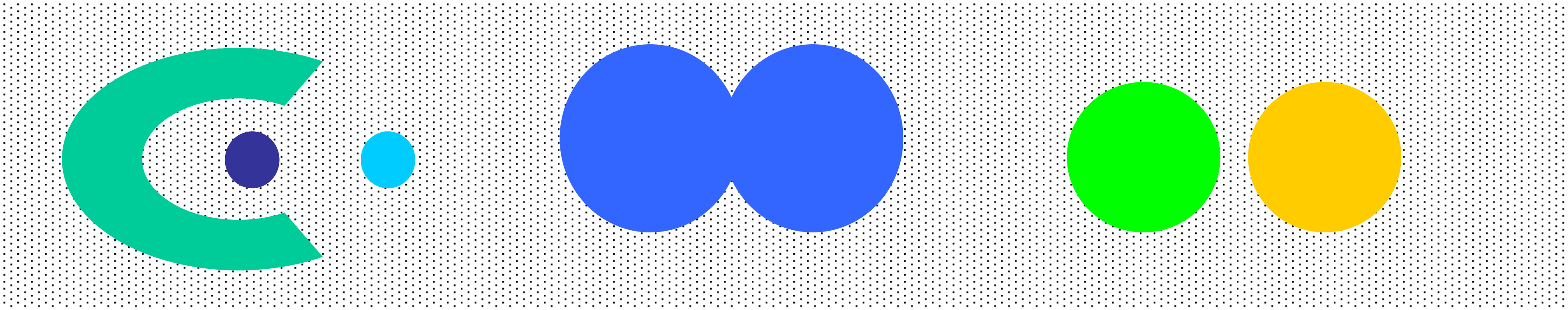
8 contiguous clusters

Types of Clusters: Density-Based

Density-based

A cluster is a dense region of points, which is separated by low-density regions, from other regions of high density.

Used when the clusters are irregular or intertwined, and when noise and outliers are present.



6 density-based clusters

Clustering Algorithms

- K-means and its variants
- Density-based clustering
- Hierarchical clustering

Types of Clusters: Objective Function

Clusters Defined by an Objective Function

- Finds clusters that minimize or maximize an objective function.
- Enumerate all possible ways of dividing the points into clusters and evaluate the `goodness' of each potential set of clusters by using the given objective function. (NP Hard)
- Can have global or local objectives.
  - Hierarchical clustering algorithms typically have local objectives
  - Partitional algorithms typically have global objectives

K-means Clustering

- **Partitional clustering** approach
- Each cluster is associated with a centroid (center point)
- Each point is assigned to the cluster with the **closest centroid**
- **Number of clusters**, K, must be specified
- The basic algorithm is very simple

```
Select K points as the initial centroids.
repeat
    Form K clusters by assigning all points to the closest centroid.
    Recompute the centroid of each cluster.
until The centroids don't change
```

K-means Clustering – Details

- Initial centroids are often chosen **randomly**.
  - Clusters produced vary from one run to another.
- The centroid is (typically) the **mean** of the points in the cluster.
- '**Closeness**' is measured by Euclidean distance, cosine similarity, correlation, etc.
- K-means will converge for common similarity measures mentioned above.
- Most of the convergence happens in the first **few iterations**.
  - Often the stopping condition is changed to 'Until relatively few points change clusters'
- **Complexity** is O(n * K * I * d)
  - n = number of points, K = number of clusters,
    I = number of iterations, d = number of attributes

Evaluating K-means Clusters

Most common measure is **Sum of Squared Error** (SSE) or **inertia** metric

- For each point, the **error** is the distance to the nearest cluster
- To get SSE, we square these errors and sum them.

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} dist^2(m_i, x)$$

- *x* is a data point in cluster C_i and m_i is the representative point for cluster C_i can show that m_i corresponds to the center (mean) of the cluster
- Given two clusters, we can choose the one with the **smallest error**
- One **easy way** to reduce SSE is to increase K, the number of clusters

  A good clustering with smaller K can have a lower SSE than a poor clustering with higher K

K-means Centroid Update

Using the Euclidean distance we have:

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} (x - m_i)^2$$

In order to minize SSE w.r.t. m_k:

$$\frac{\partial SSE}{\partial m_k} = \frac{\partial}{\partial m_k} \sum_{i=1}^{K} \sum_{x \in C_i} (x - m_i)^2 =$$

$$\sum_{i=1}^{K} \sum_{x \in C_i} \frac{\partial}{\partial m_k} (x - m_i)^2 = \sum_{x \in C_k} -2\,(x - m_k) = 0$$

$$\Rightarrow |C_k| m_k = \sum_{x \in C_k} x \Rightarrow m_k = \frac{1}{|C_k|} \sum_{x \in C_k} x$$

Thus the best centroid for minimizing the SSE of a cluster is the mean of the points in the cluster.

What happen if we use L_1 instead of Euclidean distance?

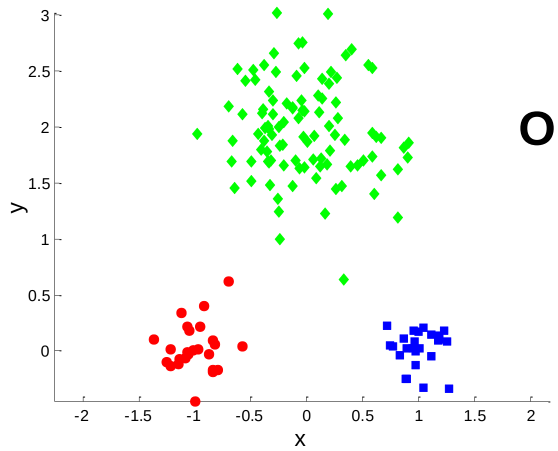
Problems with Selecting Initial Points

- If there are K 'real' clusters then the chance of randomly selecting one centroid from each cluster is small. Chance is relatively small when K is large
- If clusters have the same size, n, then

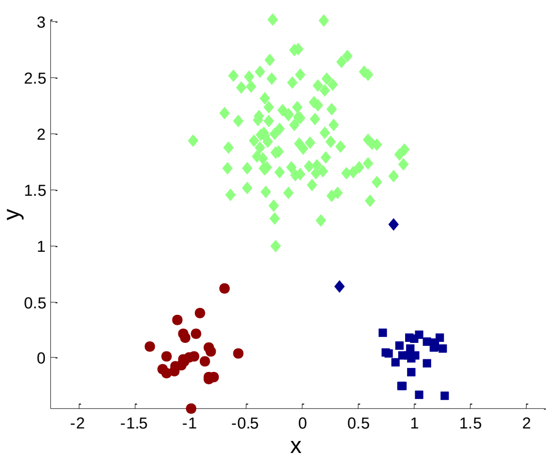
$$P = \frac{\text{number of ways to select one centroid from each cluster}}{\text{number of ways to select } K \text{ centroids}} = \frac{K!n^K}{(Kn)^K} = \frac{K!}{K^K}$$

- For example, if K = 10, then probability = $10!/10^{10}$ = 0.00036
- Sometimes the initial centroids will **readjust themselves** in 'right' way, and sometimes they don't
- Consider an example of five pairs of clusters

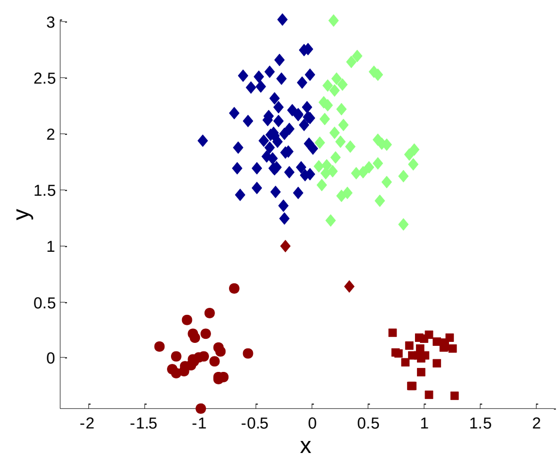
Two different K-means Clusterings



Original Points

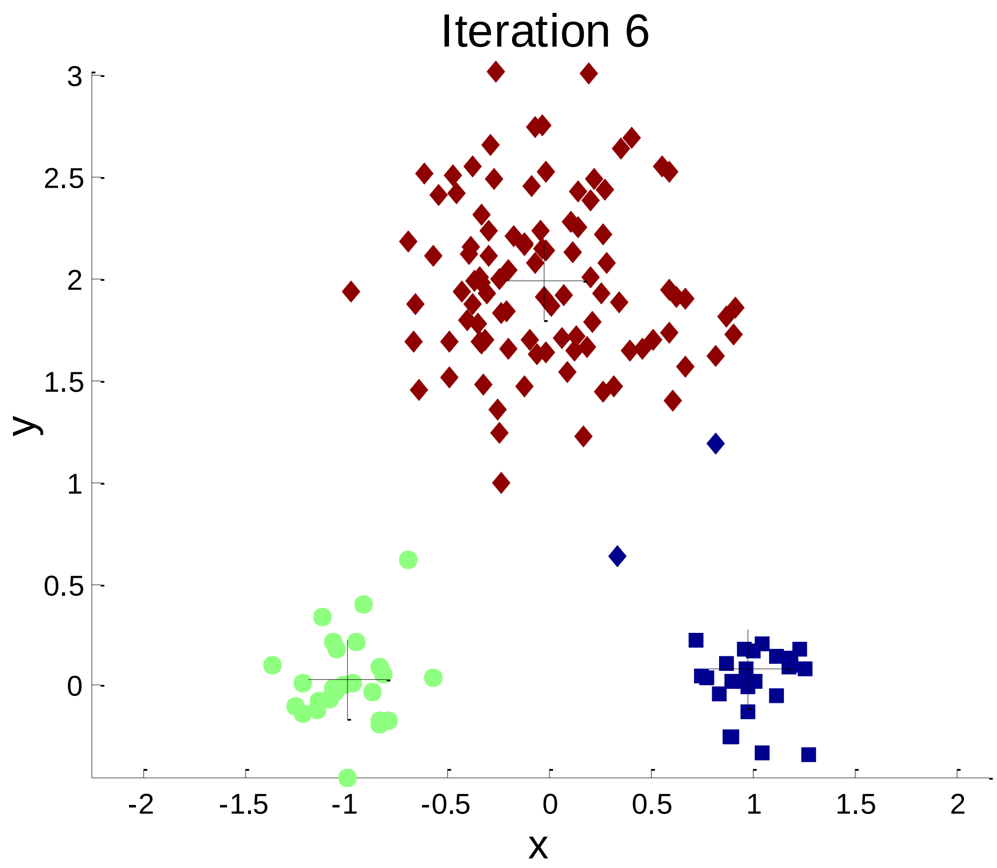


Optimal Clustering

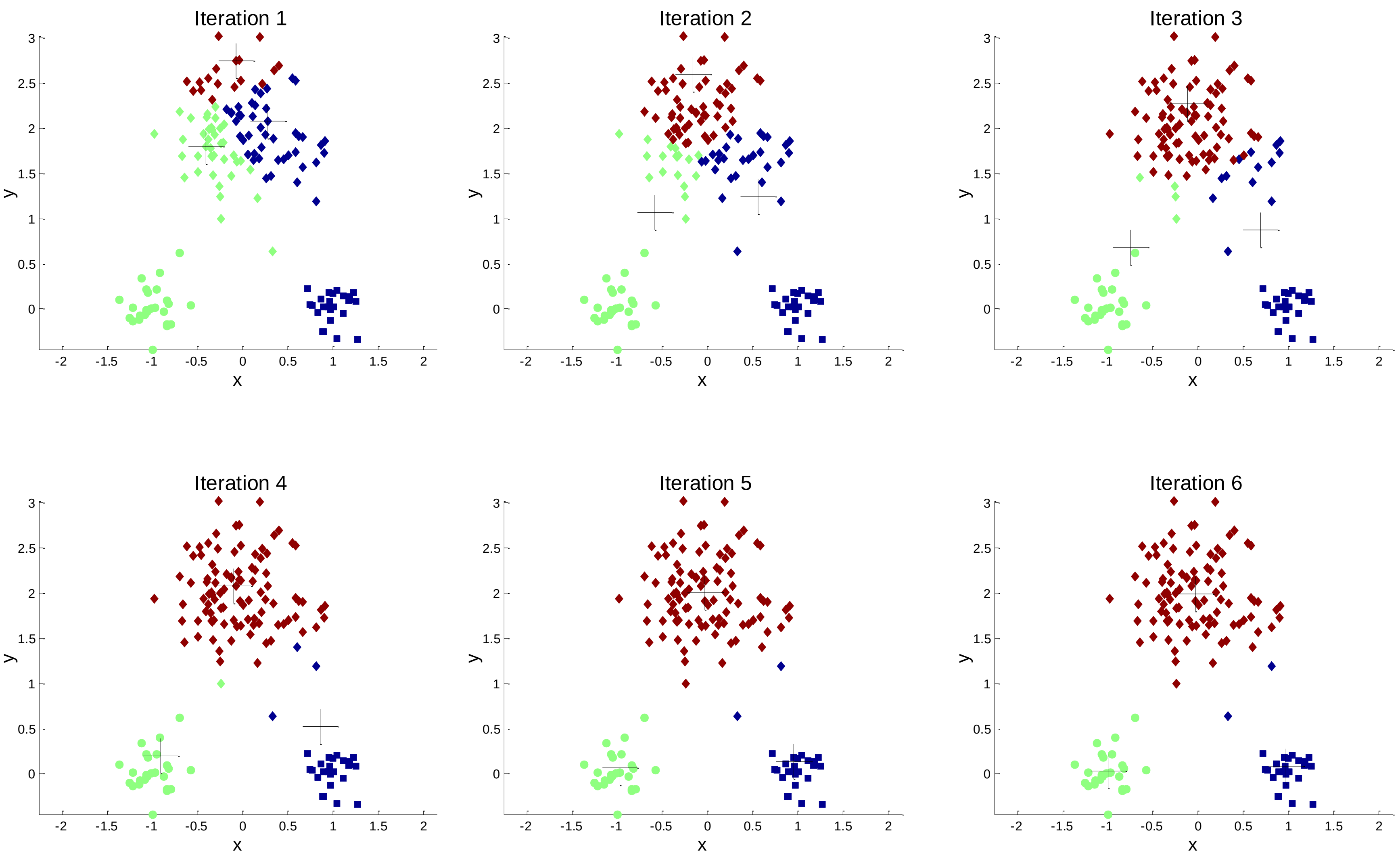


Sub-optimal Clustering

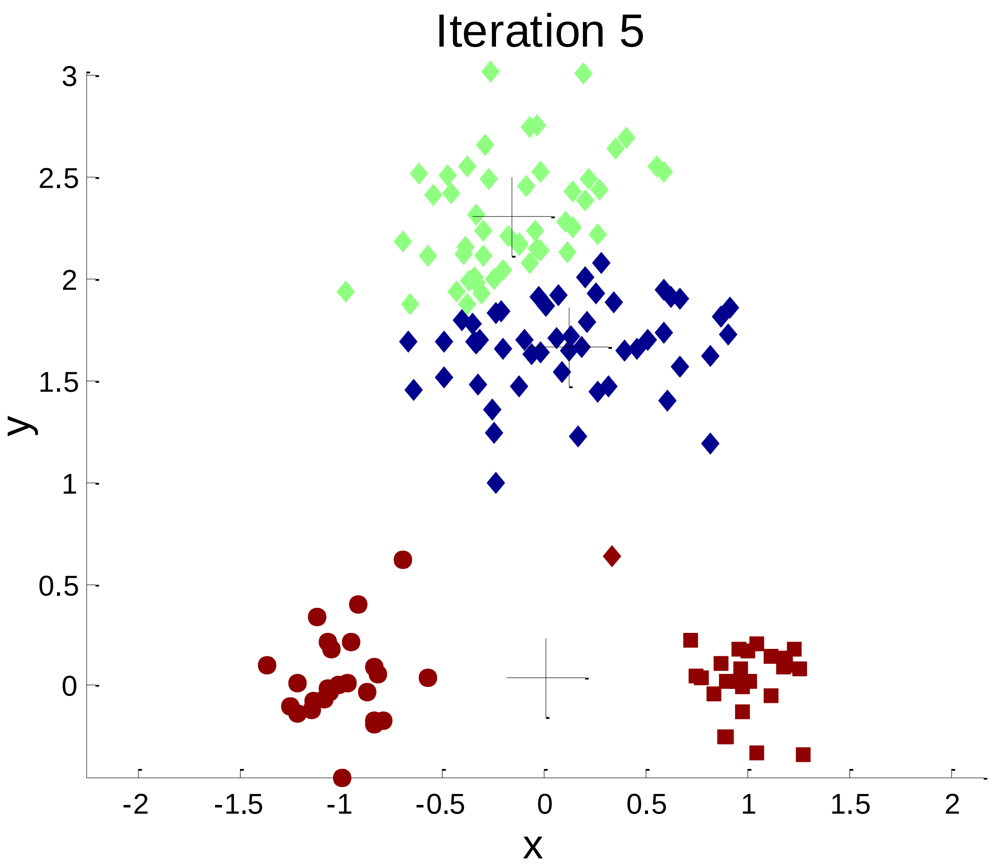
Importance of Choosing Initial Centroids



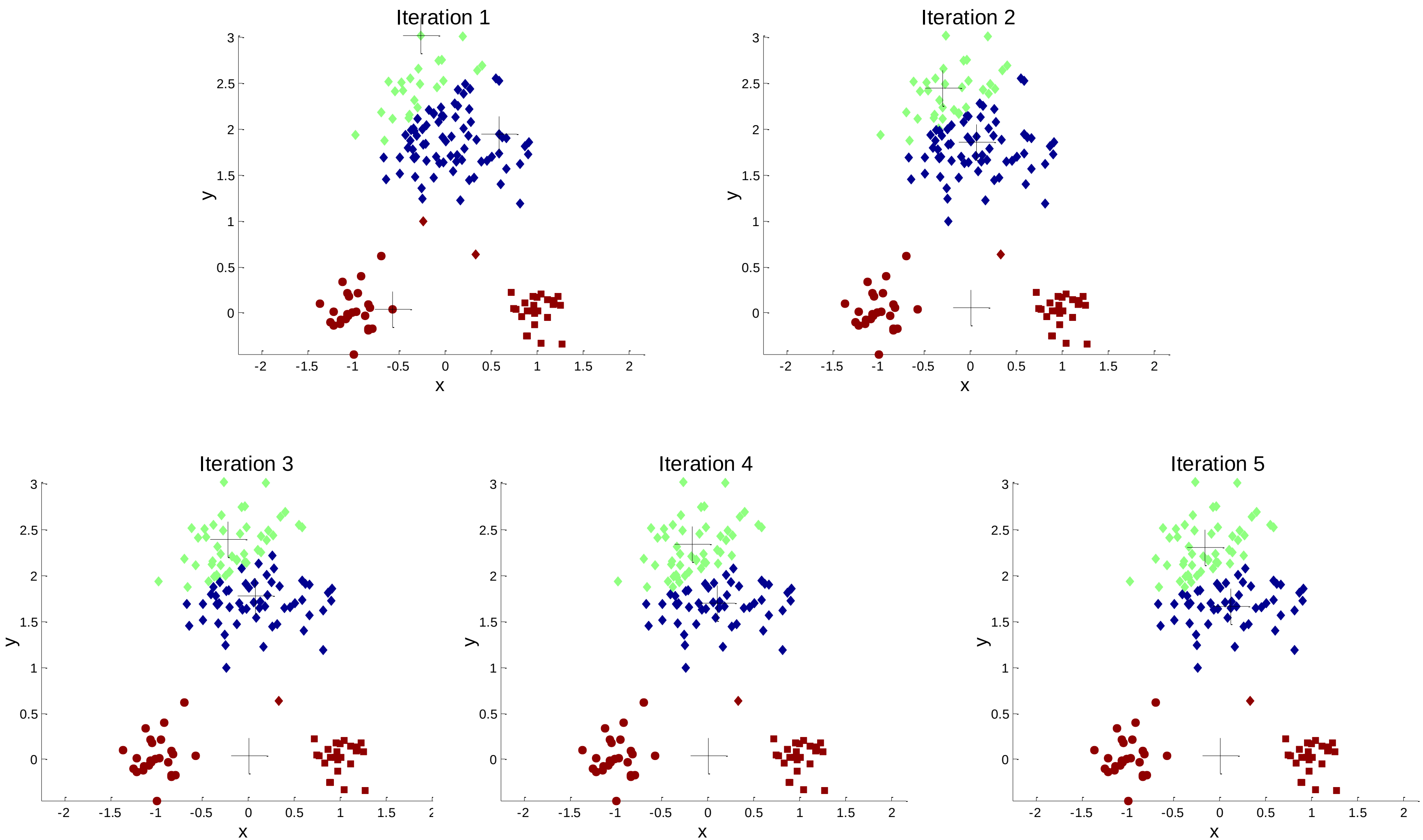
Importance of Choosing Initial Centroids



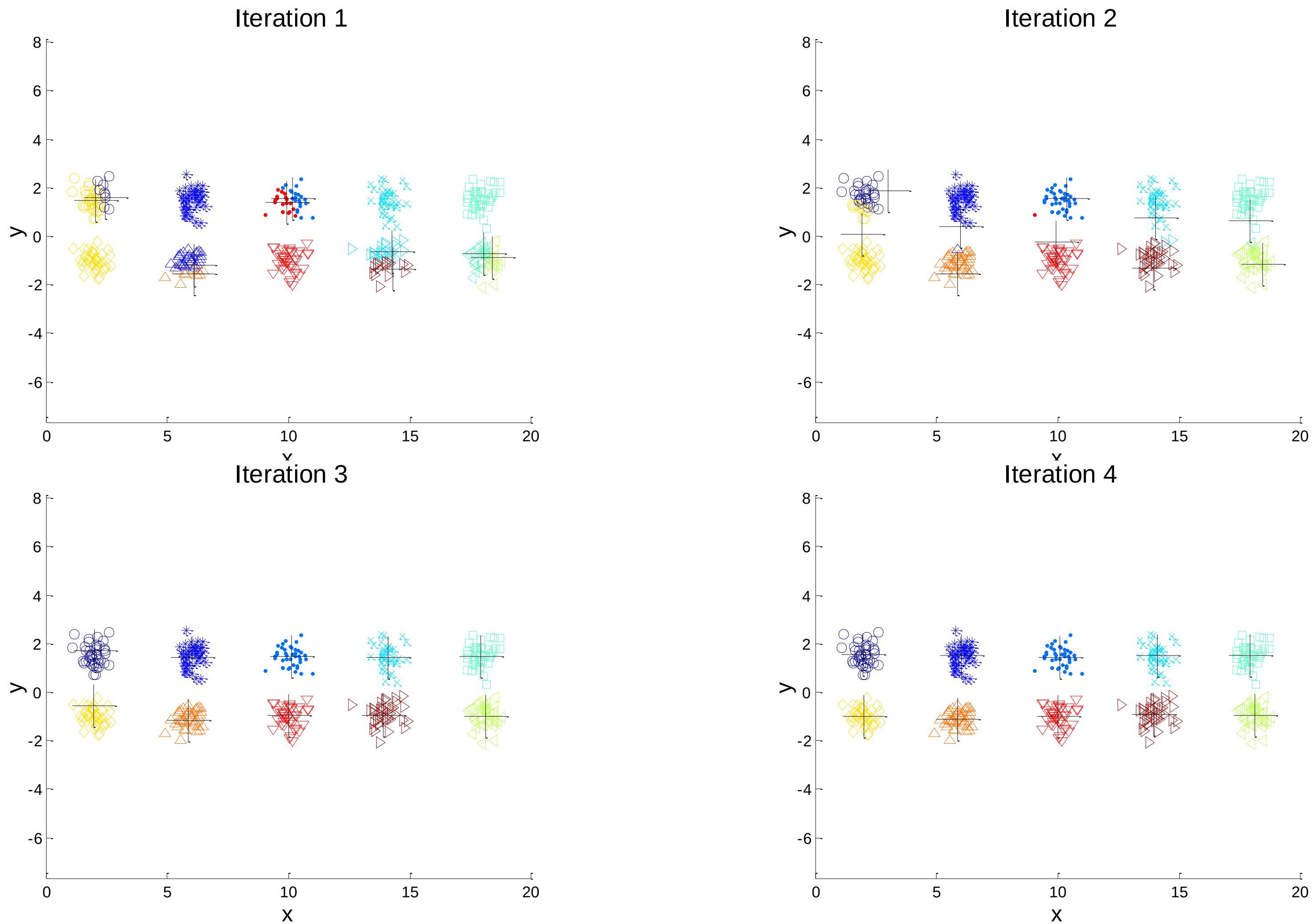
Importance of Choosing Initial Centroids …



Importance of Choosing Initial Centroids …

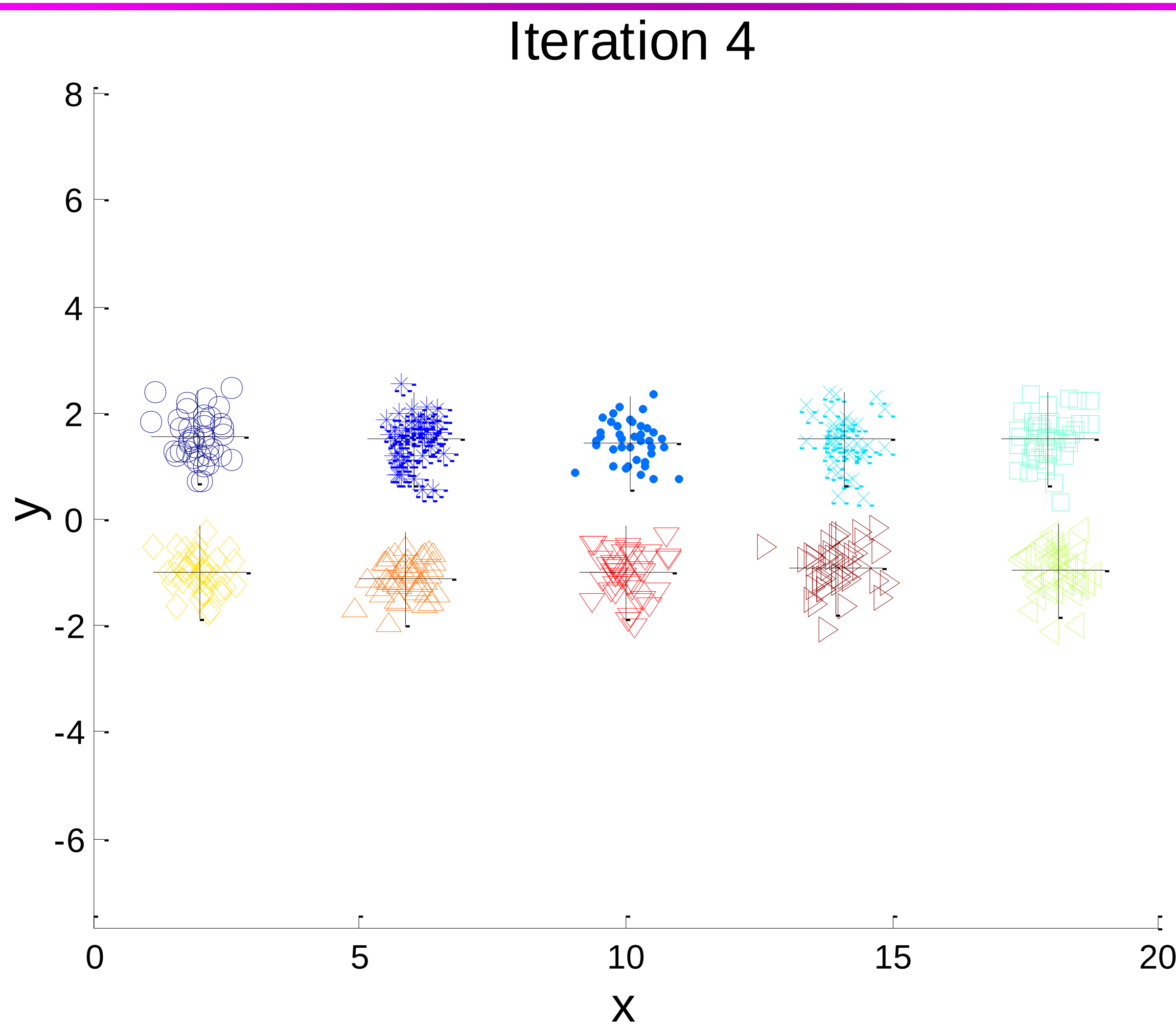


10 Clusters Example



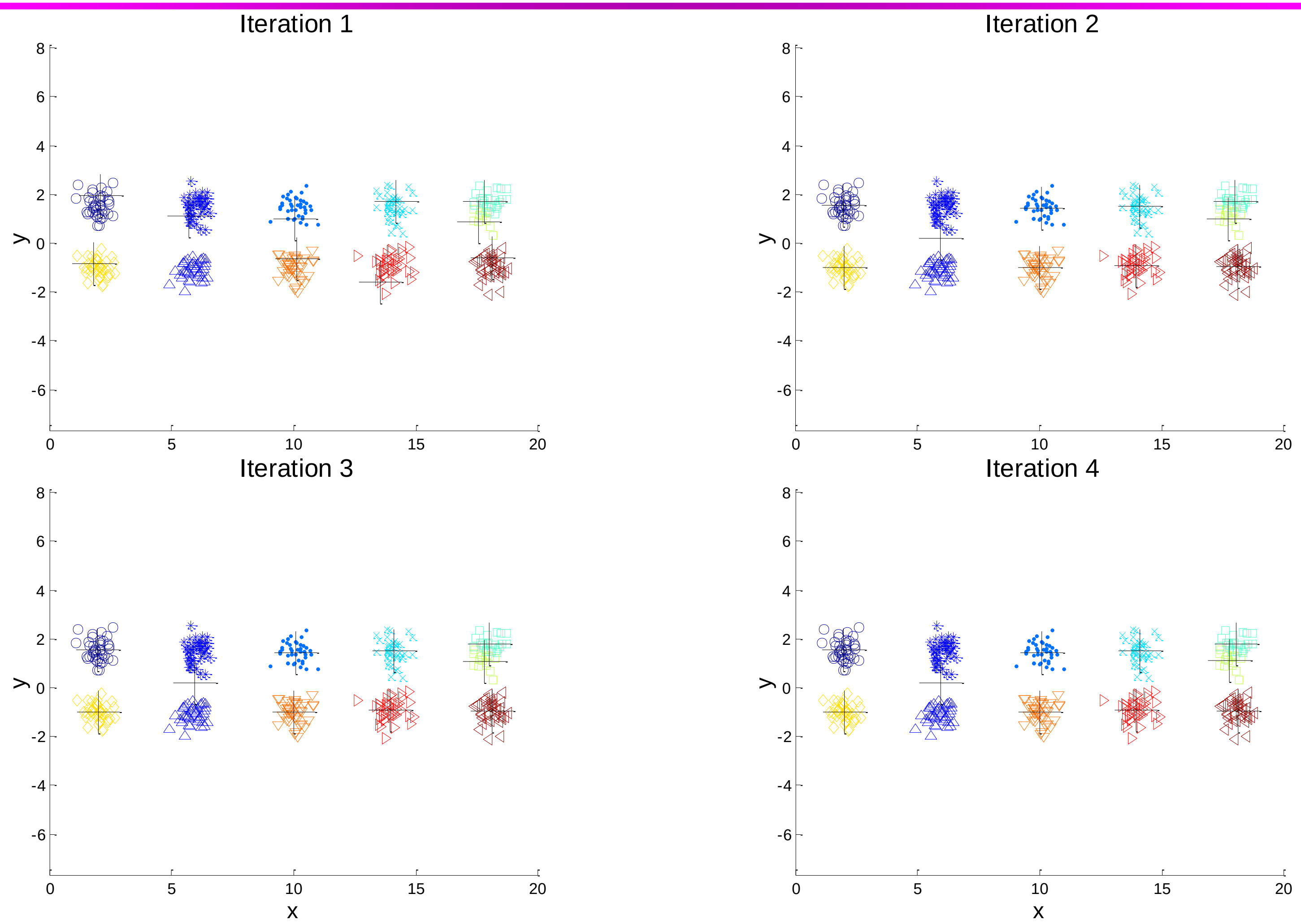
Starting with two initial centroids in one cluster of each pair of clusters

10 Clusters Example



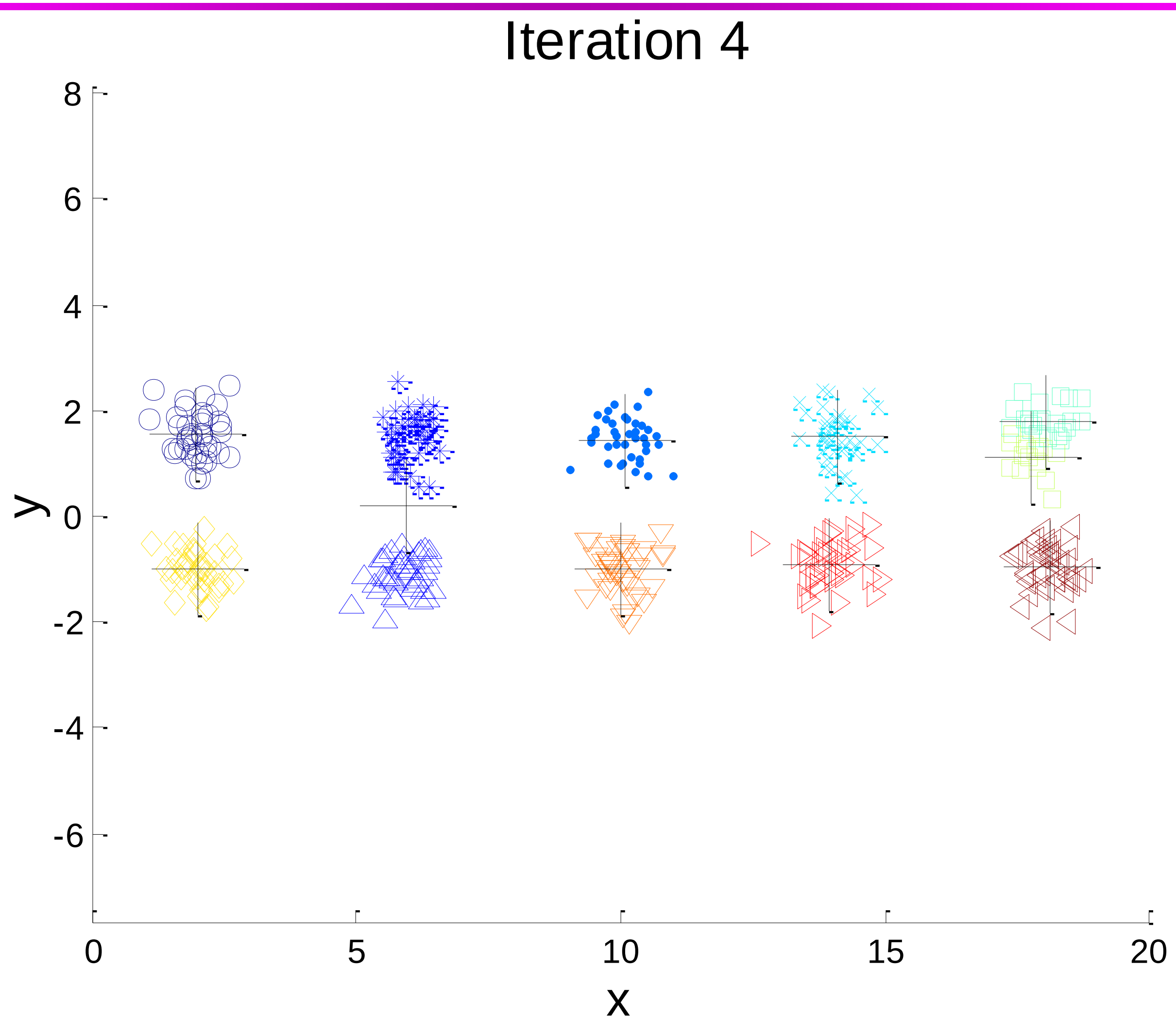
Starting with two initial centroids in one cluster of each pair of clusters

10 Clusters Example



Starting with some pairs of clusters having three initial centroids, while other have only one.

10 Clusters Example



Starting with some pairs of clusters having three initial centroids, while other have only one.

Solutions to Initial Centroids Problem

- **Multiple runs**
- Sample and **use hierarchical clustering** to determine initial centroids
- Select more than k initial centroids and then select **among** these initial centroids

  Select most widely separated
- **Postprocessing**
- **Bisecting K-means**

  Not as susceptible to initialization issues

Pre-processing and Post-processing

- Pre-processing
  - Normalize the data
  - Eliminate outliers

- Post-processing
  - Eliminate small clusters that may represent outliers
  - Split ‘loose’ clusters, i.e., clusters with relatively high SSE
  - Merge clusters that are ‘close’ and that have relatively low SSE

Limitations of K-means

- K-means has problems when clusters are of **differing**

  - Sizes

  - Densities

  - Non-globular shapes

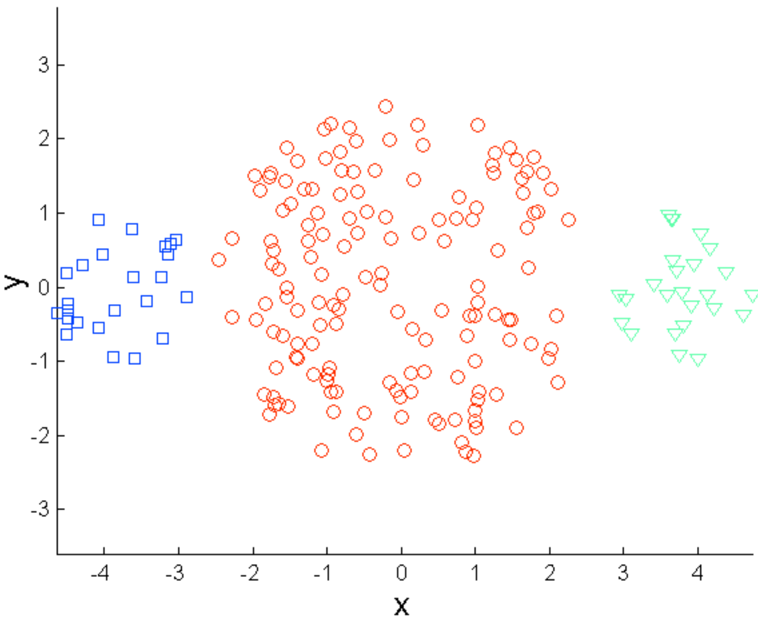
- K-means has problems when the data contains **outliers**.
- Basic K-means algorithm can yield **empty clusters**

  - Choose the point that contributes most to SSE

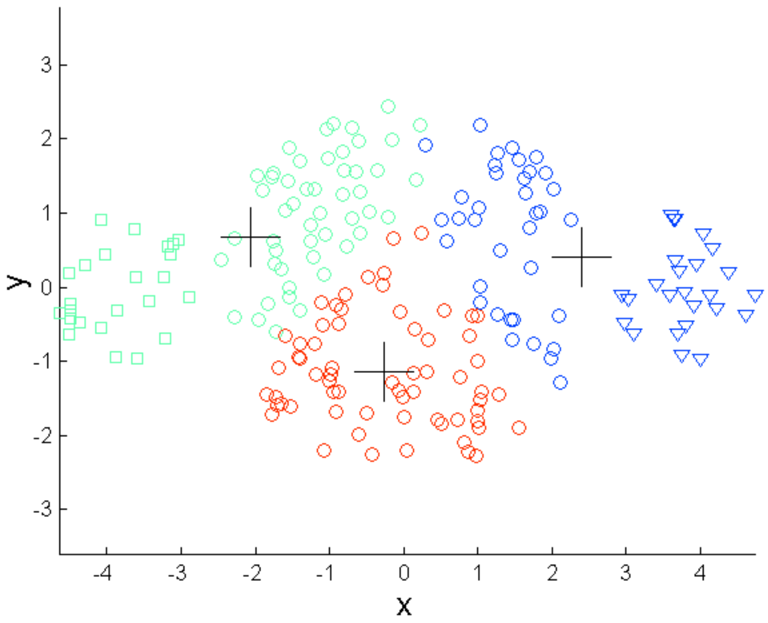
  - Choose a point from the cluster with the highest SSE

  - If there are several empty clusters, the above can be repeated several times.

Limitations of K-means: Differing Sizes

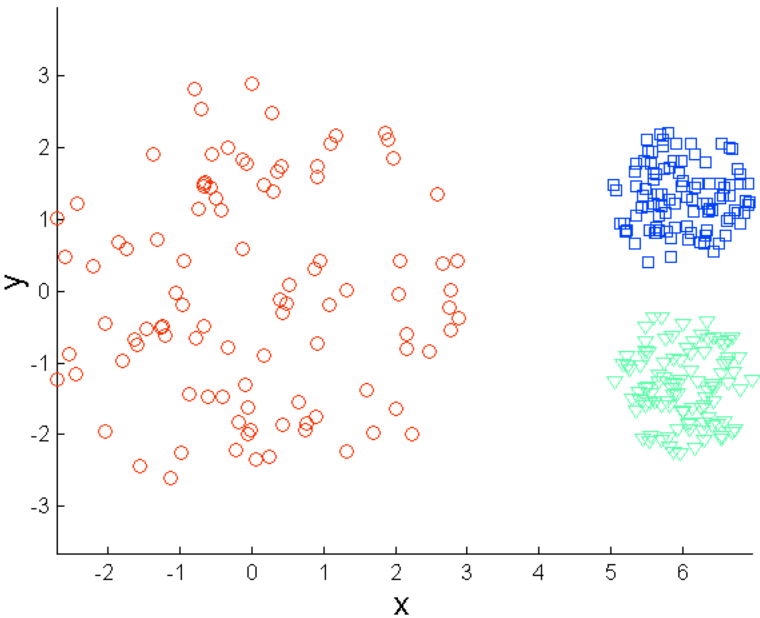


Original Points

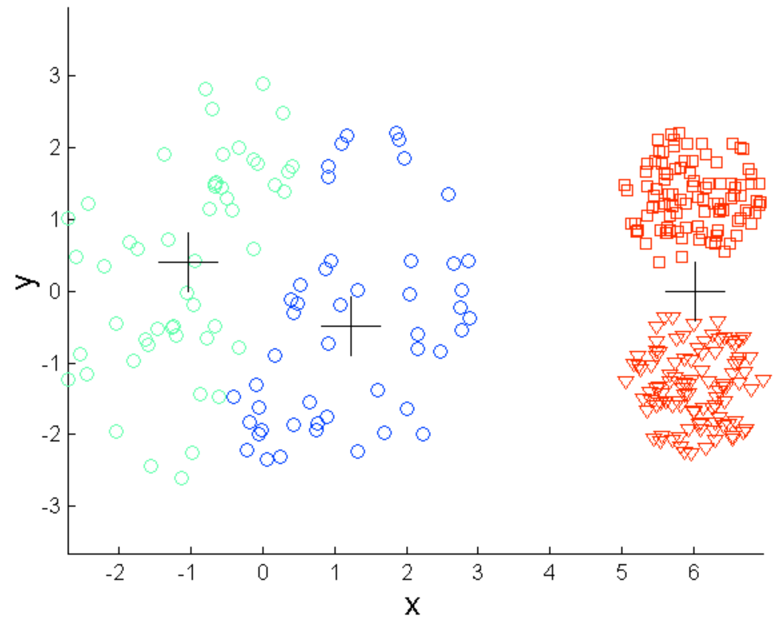


K-means (3 Clusters)

Limitations of K-means: Differing Density

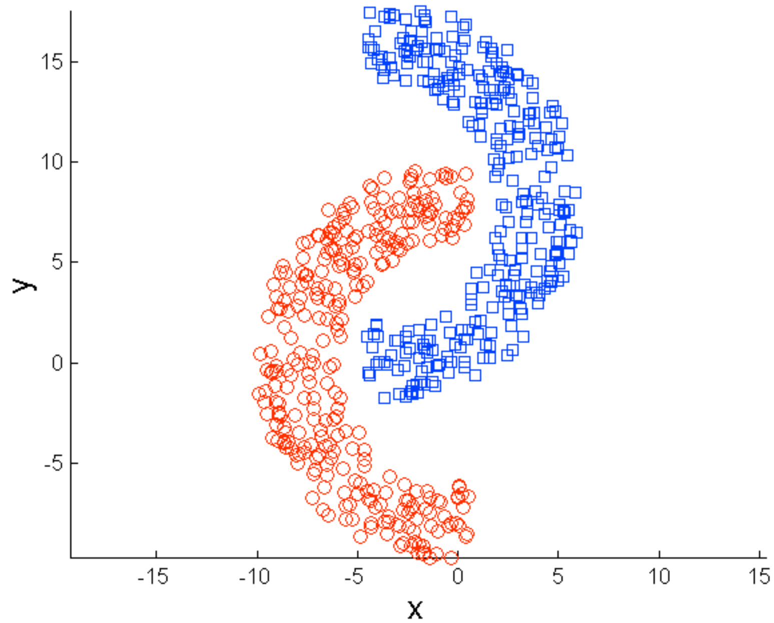


Original Points

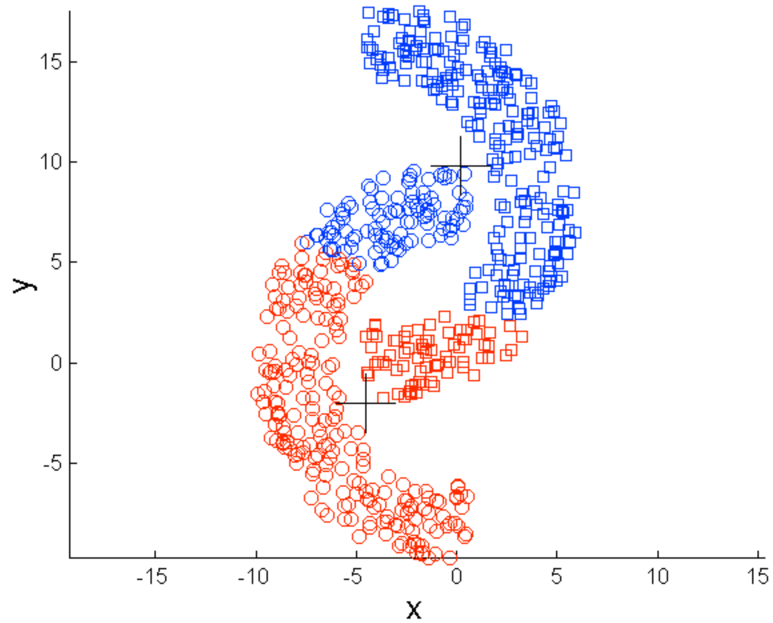


K-means (3 Clusters)

Limitations of K-means: Non-globular Shapes

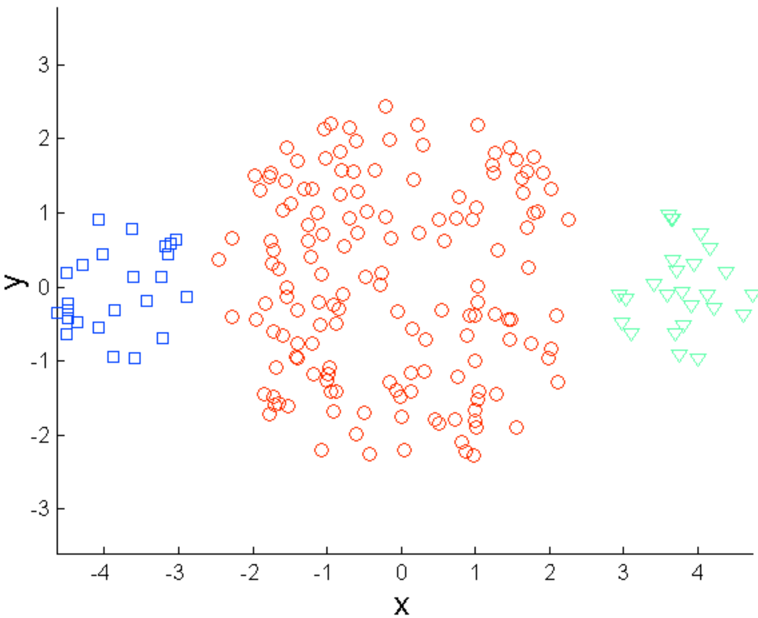


Original Points

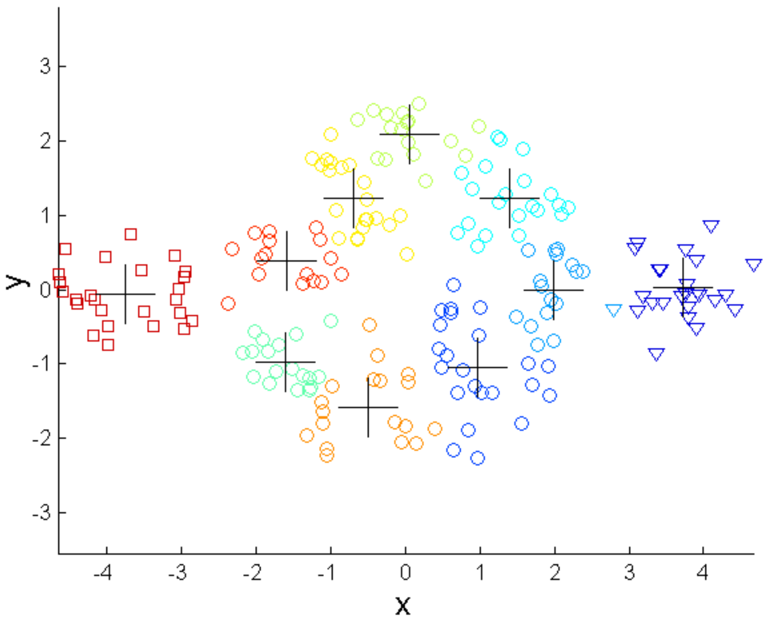


K-means (2 Clusters)

Overcoming K-means Limitations



Original Points

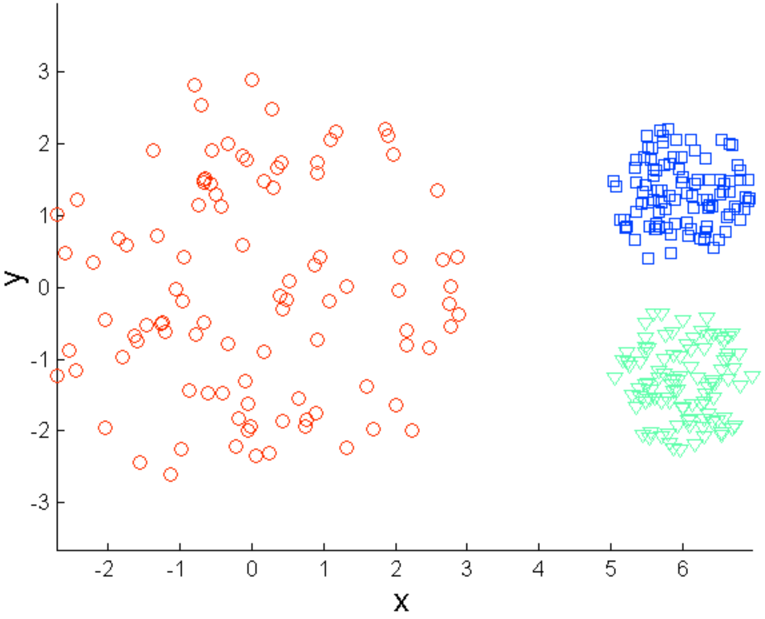


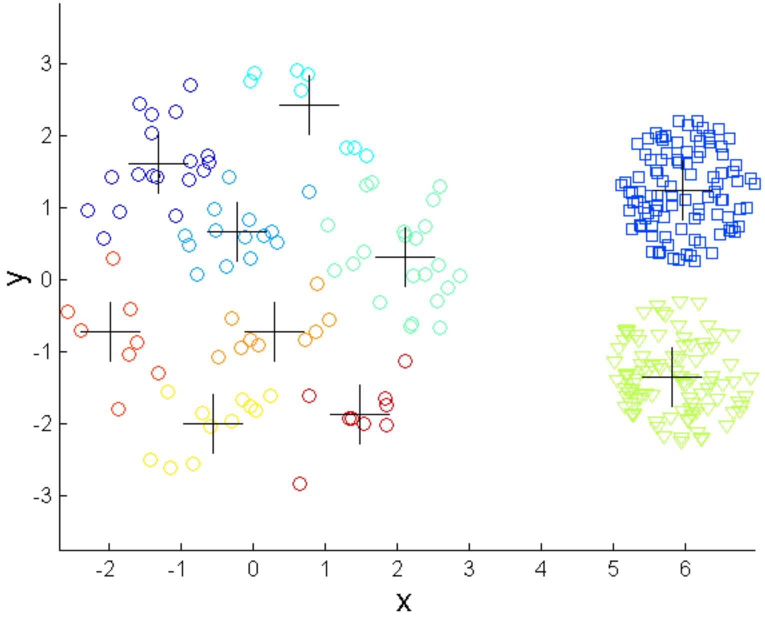
K-means Clusters

One solution is to use many clusters.

Find parts of clusters, but need to put together.

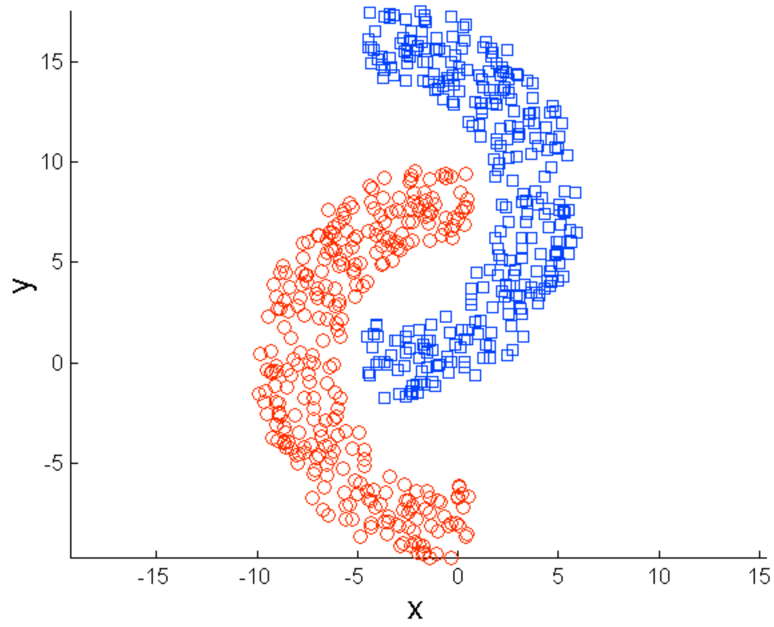
Overcoming K-means Limitations



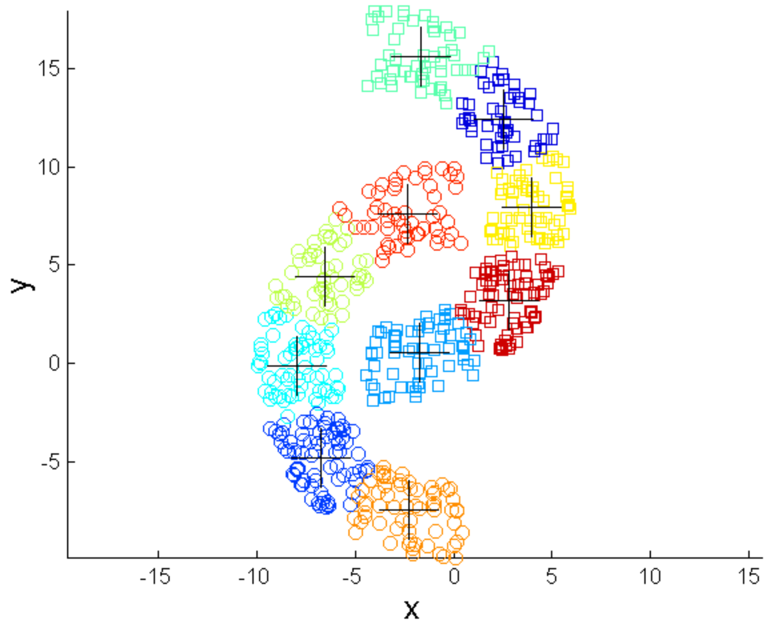


Original Points **K-means Clusters**

Overcoming K-means Limitations



Original Points



K-means Clusters

KMeans

*class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++', n_init=10, max_iter=300, tol=0.0001, precompute_distances='deprecated', verbose=0, random_state=None, copy_x=True, n_jobs='deprecated', algorithm='auto')*

- **n_clusters** int, default=8
  The number of clusters to form as well as the number of centroids to generate.
- **init** {'k-means++', 'random'},
  callable or array-like of shape (n_clusters, n_features), default='k-means++'
  Method for initialization
- **n_init** int, default=10
  Number of time the k-means algorithm will be run with different centroid seeds. The final results will be the best output of n_init consecutive runs in terms of inertia.
- **max_iter** int, default=300
  Maximum number of iterations of the k-means algorithm for a single run

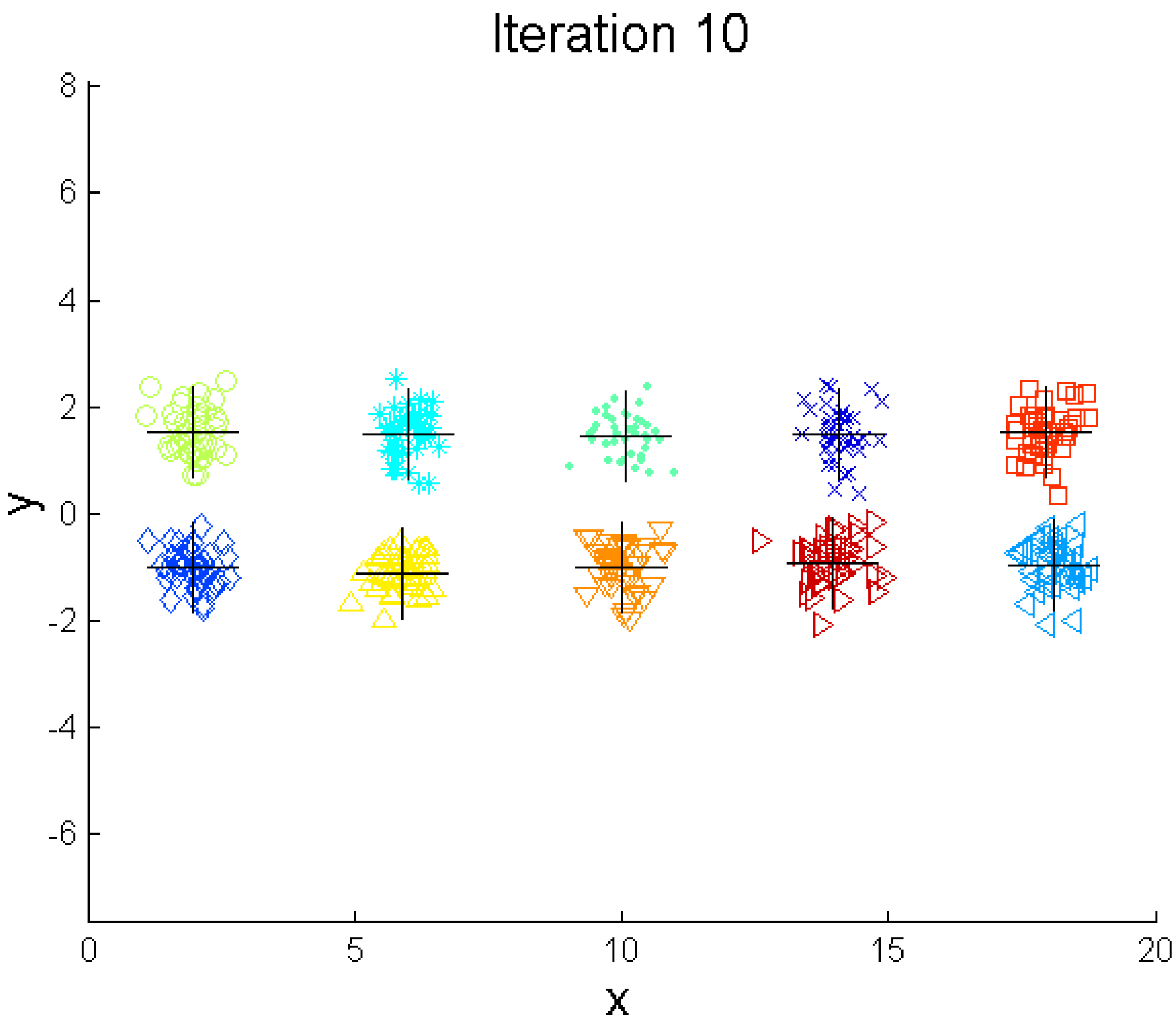
Bisecting K-means

Bisecting K-means algorithm

Variant of K-means that can produce a partitional or a hierarchical clustering

```
Initialize the list of clusters to contain the cluster containing all points.
repeat
   Select a cluster from the list of clusters
   for i = 1 to number_of_iterations do
      Bisect the selected cluster using basic K-means
   end for
   Add the two clusters from the bisection with the lowest SSE to the list of clusters.
until Until the list of clusters contains K clusters
```

Bisecting K-means Example



Density-Based Algorithms

DBSCAN

DBSCAN is a **density-based algorithm**.

- Density = number of points within a specified radius (*Eps*)

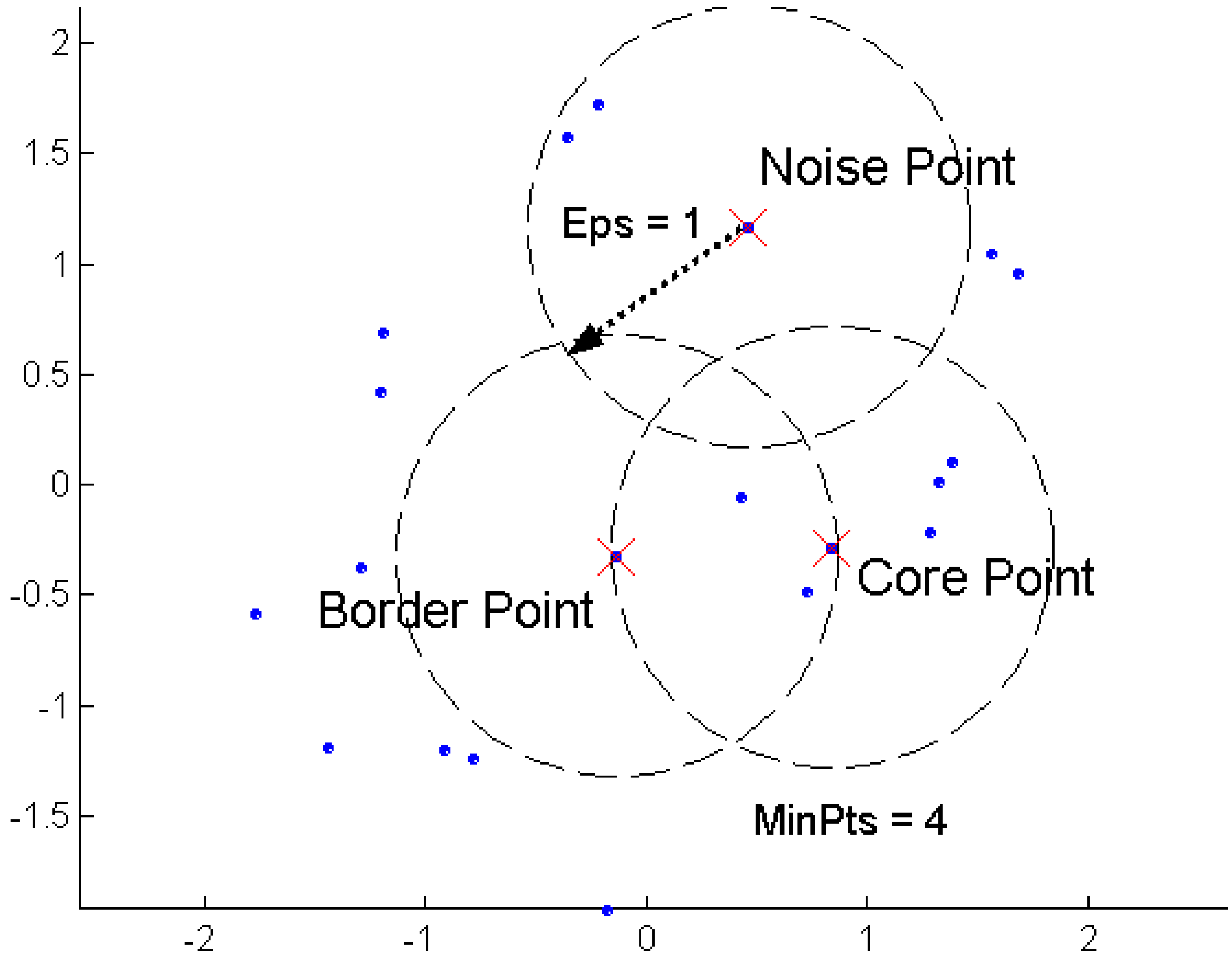
- A point is a core point if it has more than a specified number of points (*MinPts*) within *Eps*

  These are points that are at the interior of a cluster

- A border point has fewer than *MinPts* within *Eps*, but is in the neighborhood of a core point

- A noise point is any point that is not a core point or a border point.

DBSCAN: Core, Border, and Noise Points



DBSCAN Algorithm

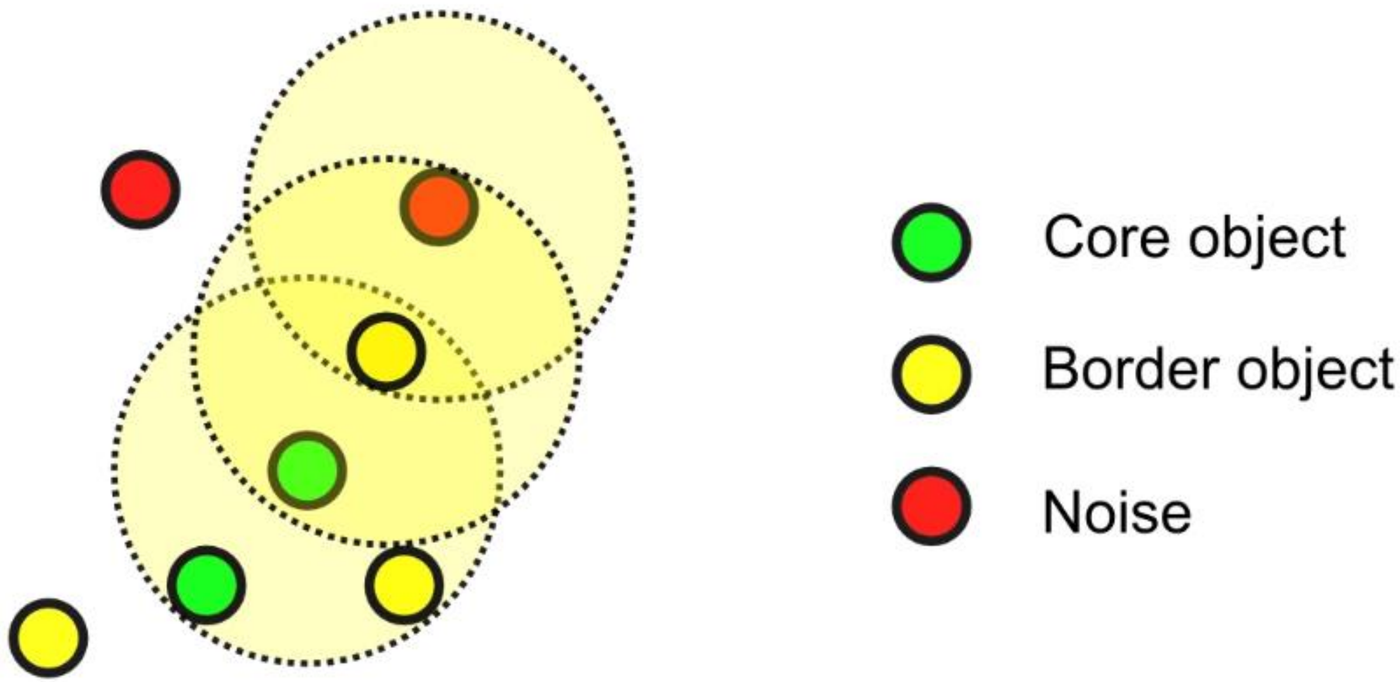
Algorithm 8.4 DBSCAN algorithm.

1: Label all points as core, border, or noise points.
2: Eliminate noise points.
3: Put an edge between all core points that are within *Eps* of each other.
4: Make each group of connected core points into a separate cluster.
5: Assign each border point to $\boxed{\text{one of the clusters of its associated core points.}}$

Border points can be neighbors of several core points/clusters
→ arbitrarily choose one!

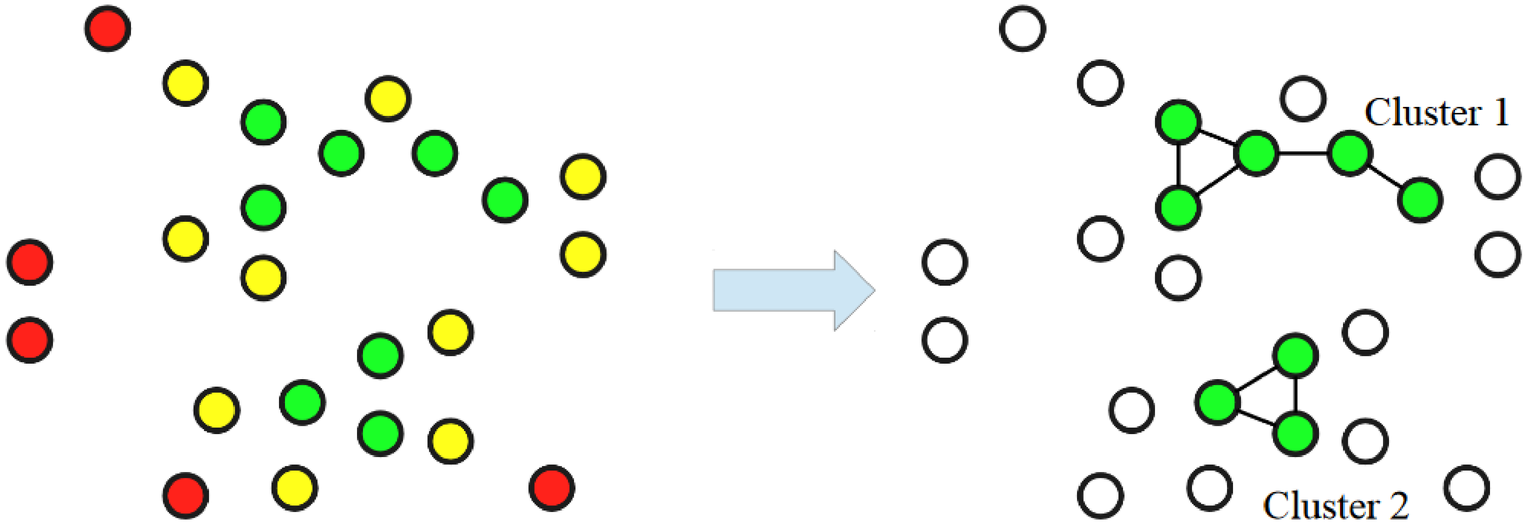
DBSCAN

Step 1: label points as **core** (dense), **border** and **noise**
Based on thresholds *Eps* (radius of neighborhood) and *MinPts* (min number of neighbors)



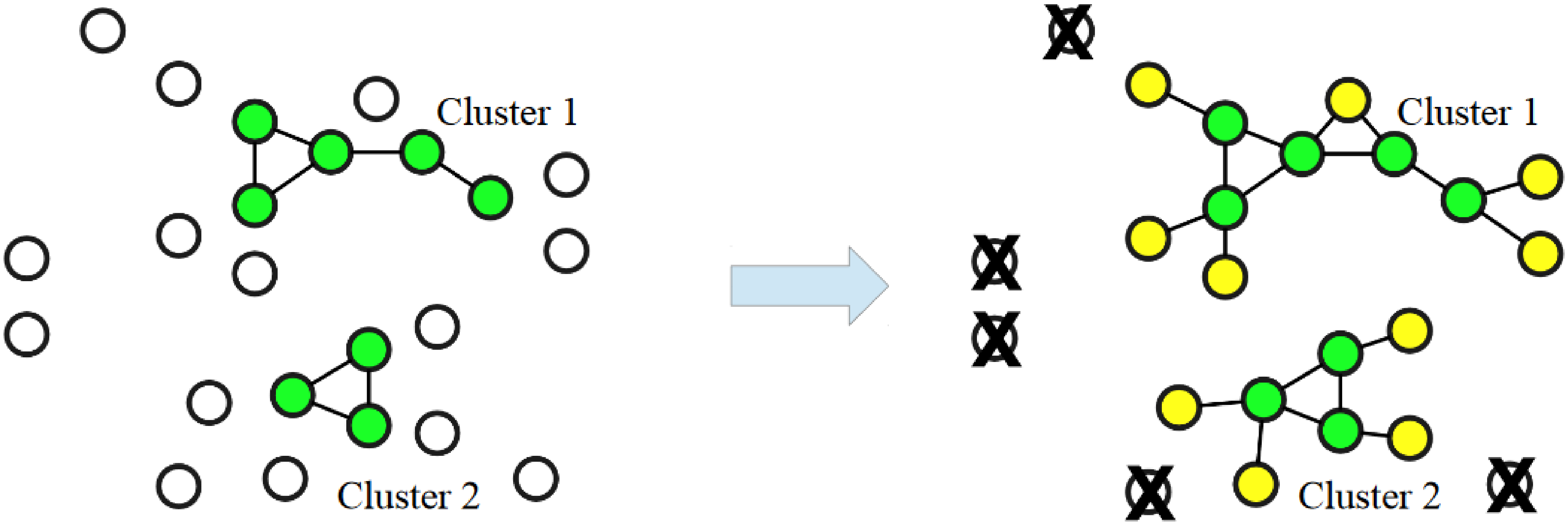
DBSCAN

Steps 3-4: **connect** *core* objects that are *neighbors*, and put them in the **same cluster**



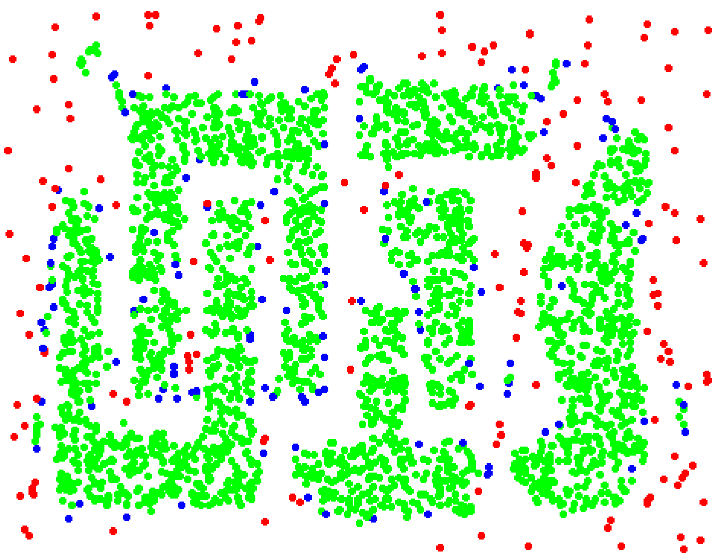
DBSCAN

Step 5: **associate** *border* objects to (one of) their core(s)



DBSCAN: Core, Border and Noise Points

Original Points

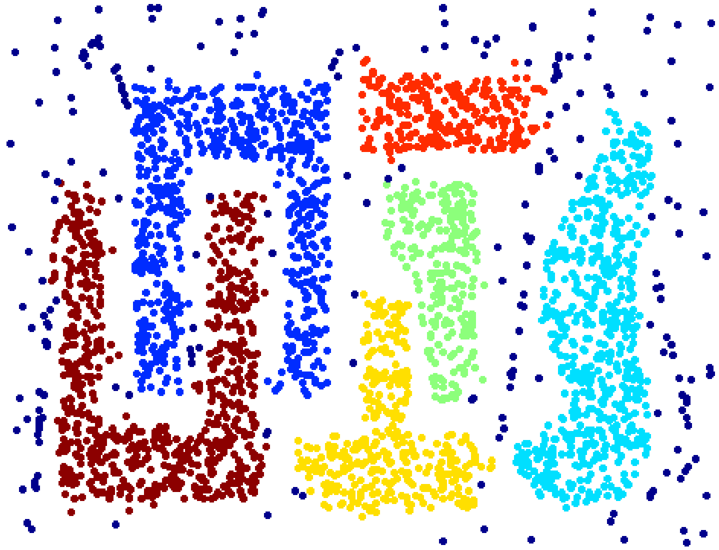


Point types: core, border and noise

Eps = 10, MinPts = 4

When DBSCAN Works Well

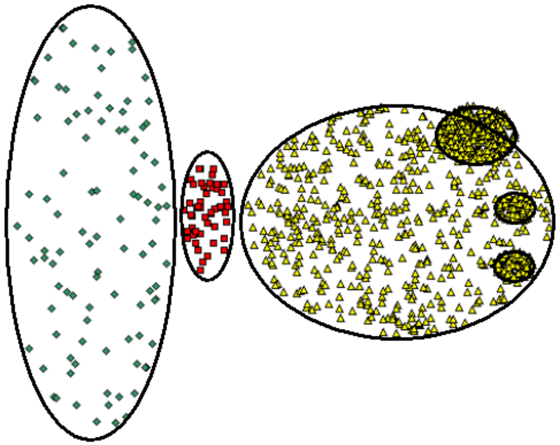
Original Points



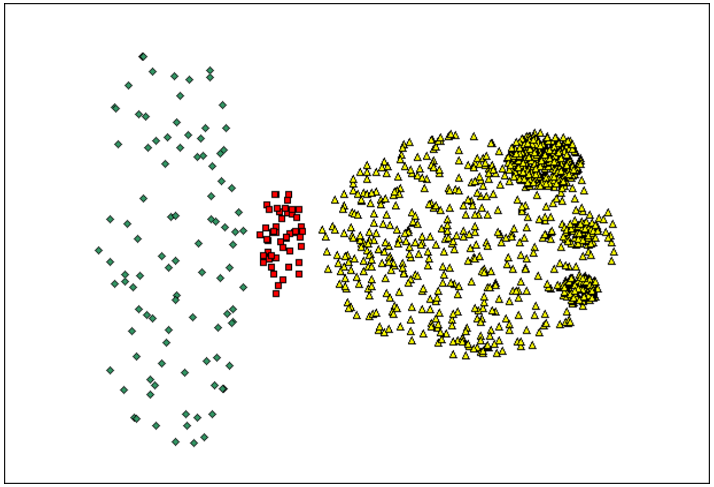
Clusters

- **Resistant to Noise**
- **Can handle clusters of different shapes and sizes**

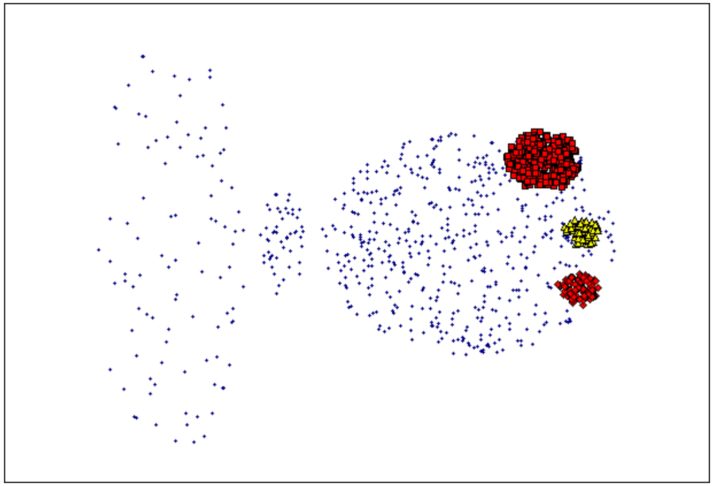
When DBSCAN Does NOT Work Well



Original Points



(MinPts=4, Eps=9.92)

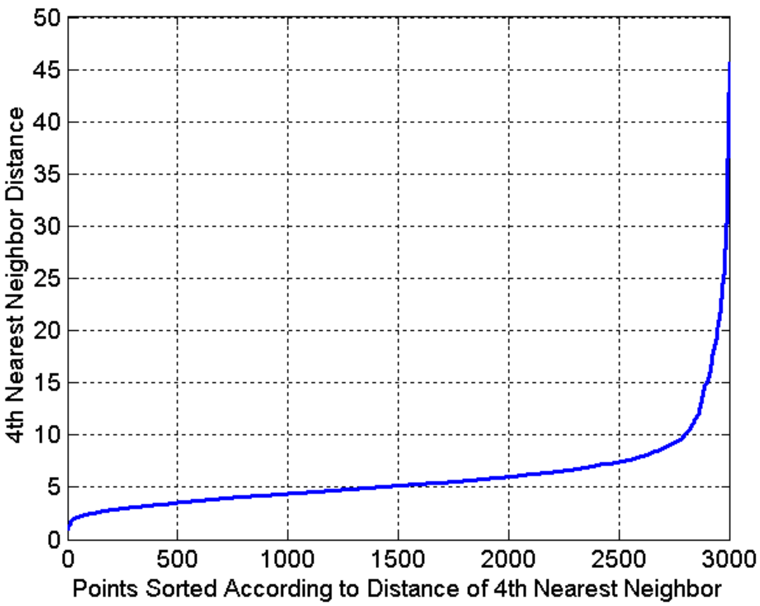


(MinPts=4, Eps=9.75).

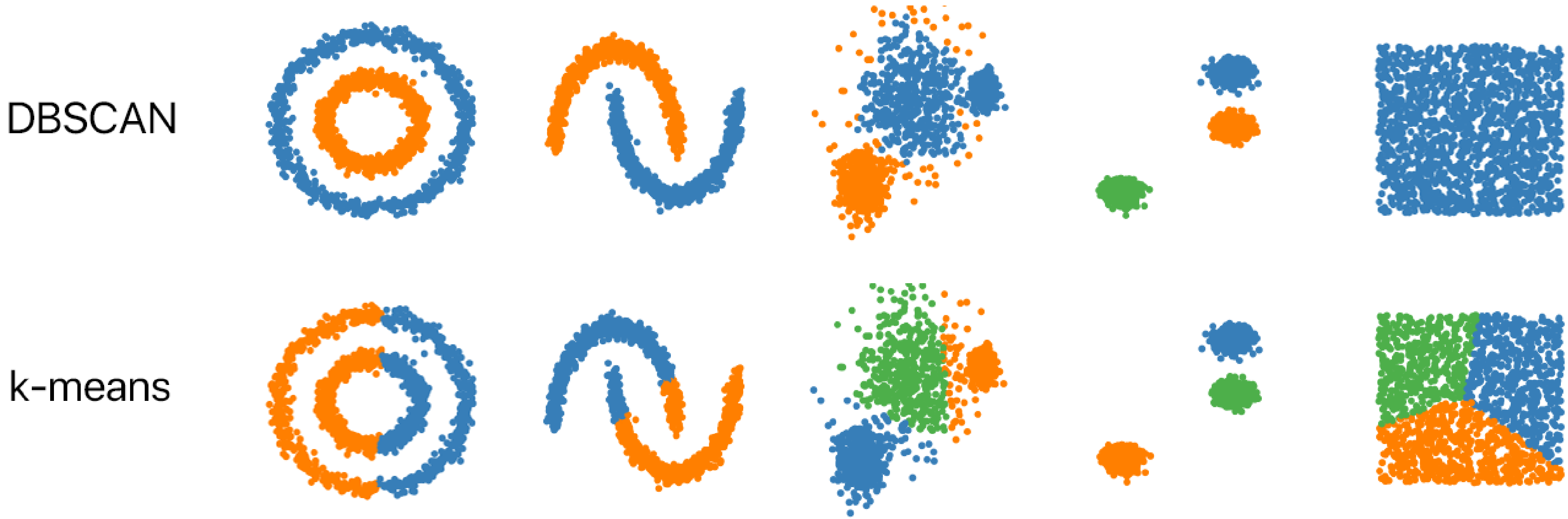
- **Varying densities**
- **High-dimensional data**

DBSCAN: Determining EPS and MinPts

- Idea is that for points in a cluster, their k^{th} nearest neighbors are at roughly the same distance
- Noise points have the k^{th} nearest neighbor at farther distance
- So, plot sorted distance of every point to its k^{th} nearest neighbor



DBSCAN vs K-means



sklearn.cluster.DBSCAN

*class [sklearn.cluster.DBSCAN](eps=0.5, *, min_samples=5, metric='euclidean', metric_params=None, algorithm='auto', leaf_size=30, p=None, n_jobs=None)*

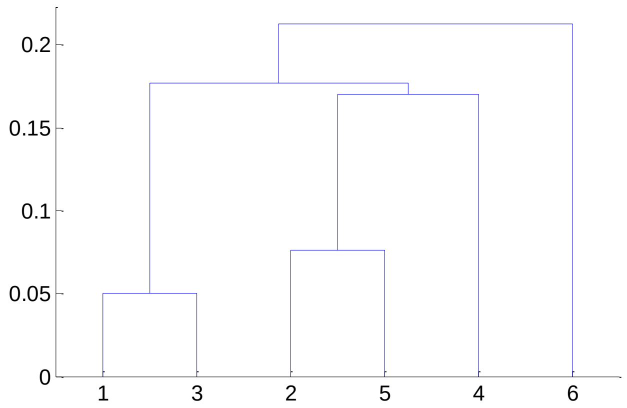
- **eps** float, default=0.5
  The maximum distance between two samples for one to be considered as in the neighborhood of the other. This is not a maximum bound on the distances of points within a cluster. This is the most important DBSCAN parameter to choose appropriately for your data set and distance function.
- **min_samples** int, default=5
  The number of samples (or total weight) in a neighborhood for a point to be considered as a core point. This includes the point itself.
- **metric** string, or callable, default='euclidean'
- **n_jobs** int, default=None
  The number of parallel jobs to run

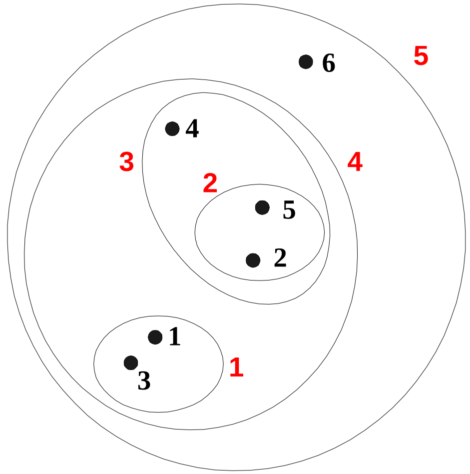
Hierarchical Clustering

Hierarchical Clustering

- Produces a set of nested clusters organized as a hierarchical tree
- Can be visualized as a dendrogram

  A tree like diagram that records the sequences of merges or splits





Strengths of Hierarchical Clustering

- Do not have to assume any particular number of clusters

  Any desired number of clusters can be obtained by 'cutting' the dendrogram at the proper level

- They may correspond to meaningful taxonomies

  Example in biological sciences (e.g., animal kingdom, phylogeny reconstruction, …)

Hierarchical Clustering

- Two main types of hierarchical clustering

  Agglomerative:

  - Start with the points as individual clusters
  - At each step, merge the closest pair of clusters until only one cluster (or k clusters) left

  Divisive:

  - Start with one, all-inclusive cluster
  - At each step, split a cluster until each cluster contains a point (or there are k clusters)

- Traditional hierarchical algorithms use a similarity or distance matrix

  Merge or split one cluster at a time

Agglomerative Clustering Algorithm

- Most popular hierarchical clustering technique

- Basic algorithm is straightforward

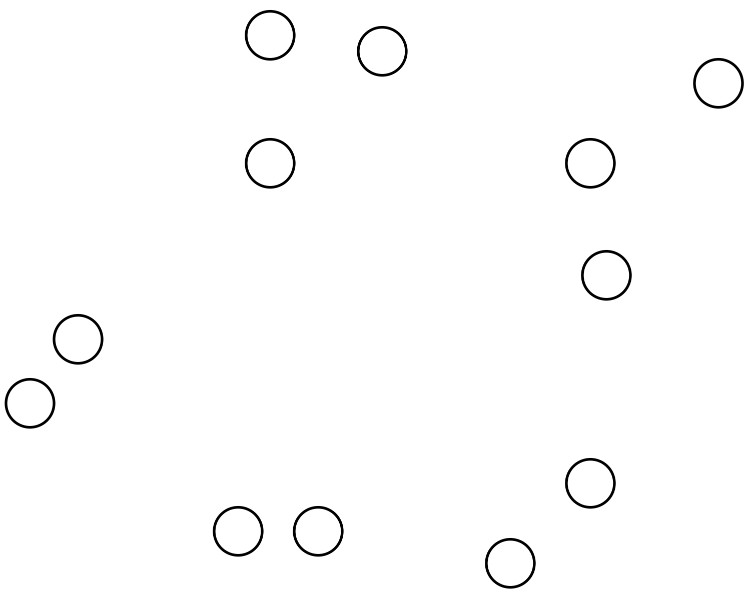
1. Compute the proximity matrix
2. Let each data point be a cluster
3. **Repeat**
4. Merge the two closest clusters
5. Update the proximity matrix
6. **Until** only a single cluster remains

- Key operation is the computation of the proximity of two clusters

Different approaches to defining the distance between clusters distinguish the different algorithms

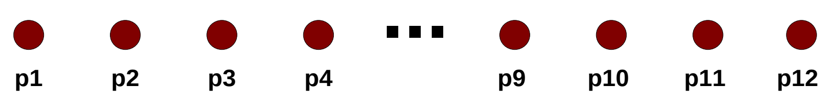
Starting Situation

Start with clusters of individual points and a distance/proximity matrix



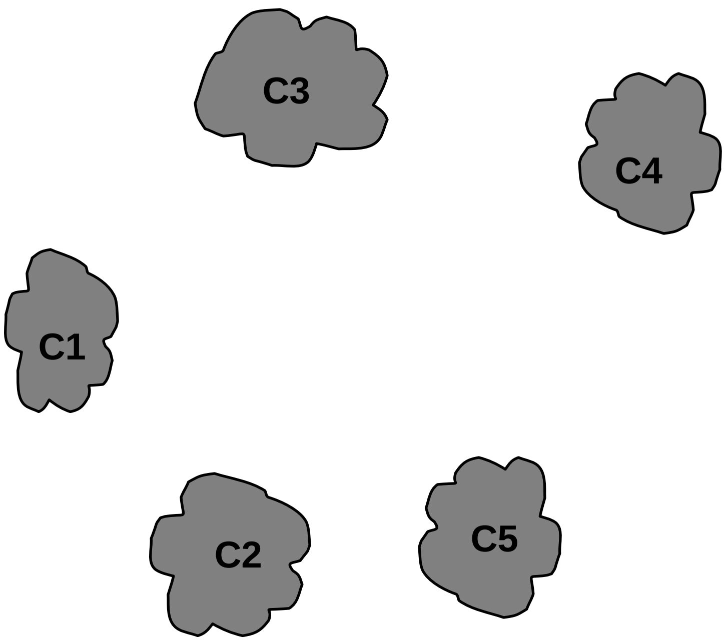
	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix



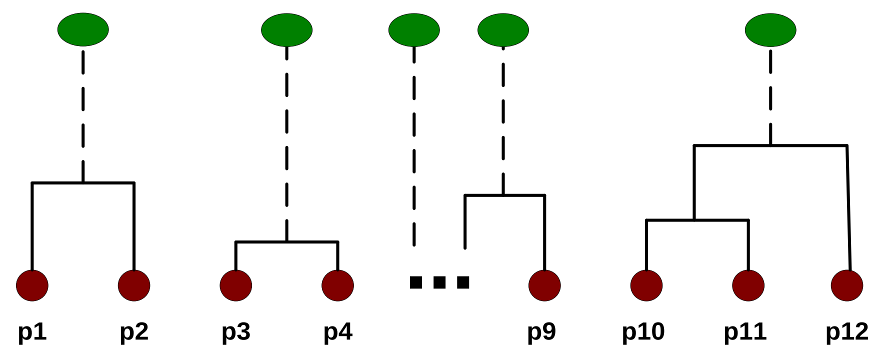
Intermediate Situation

After some merging steps, we have some clusters



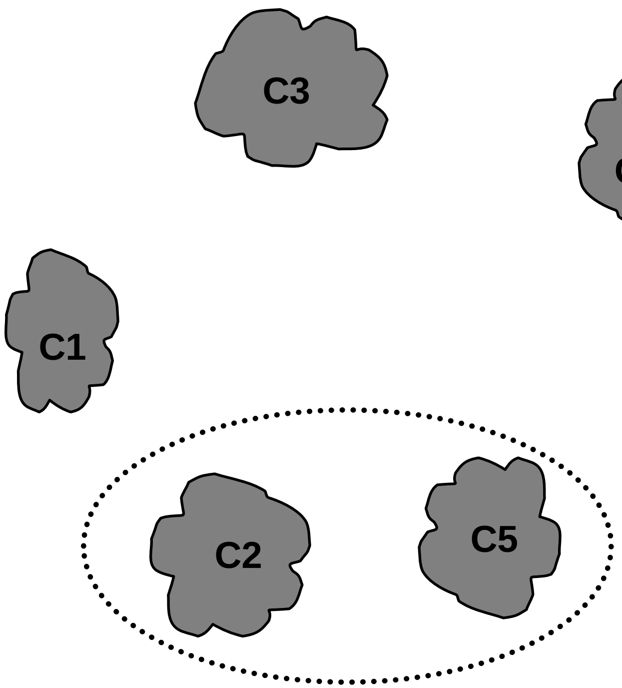
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

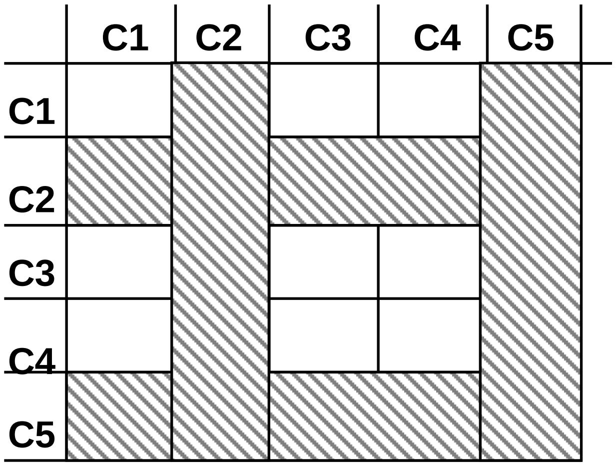
Proximity Matrix



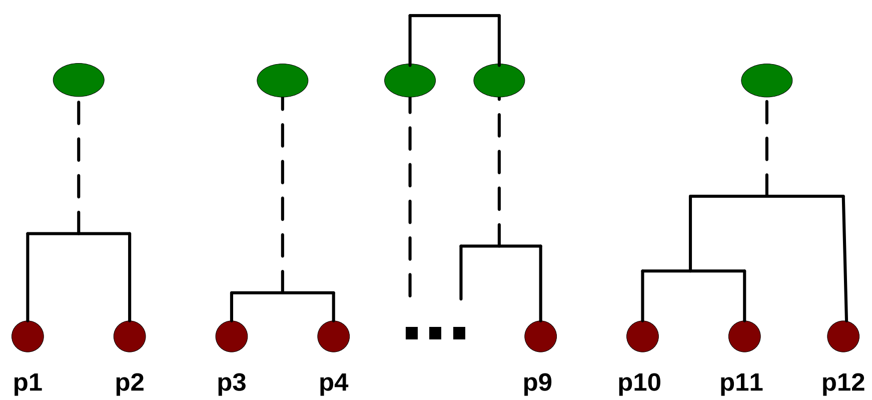
Intermediate Situation

We want to merge the two closest clusters (C2 and C5) and update the proximity matrix.



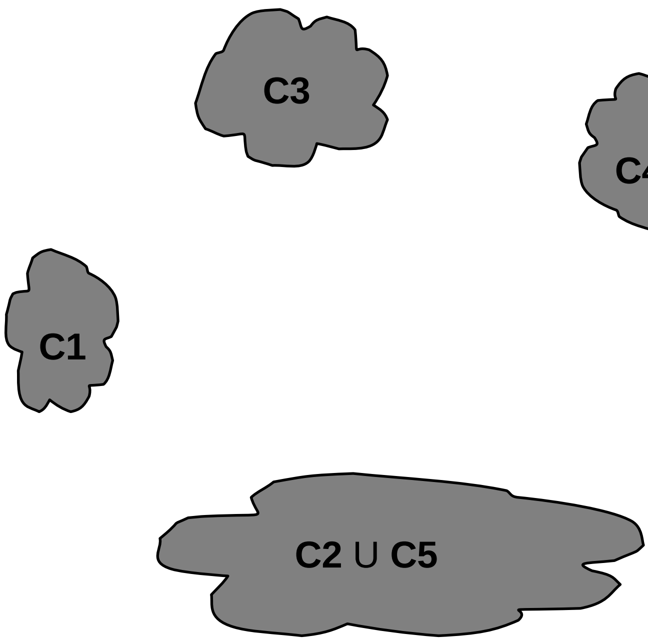


Proximity Matrix



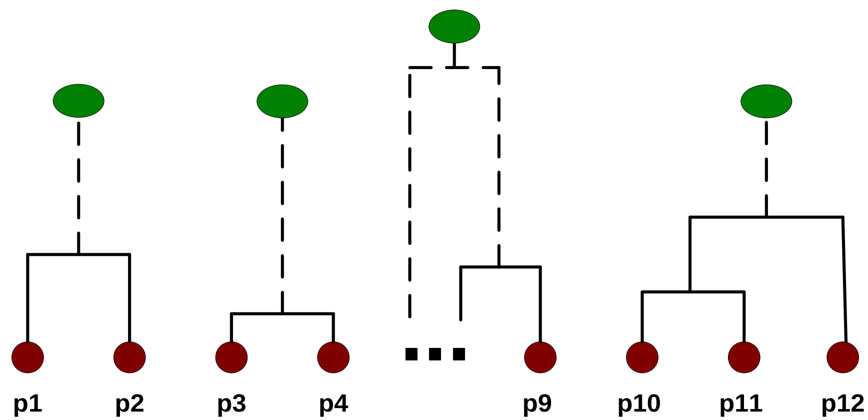
After Merging

The question is “How do we update the proximity matrix?”

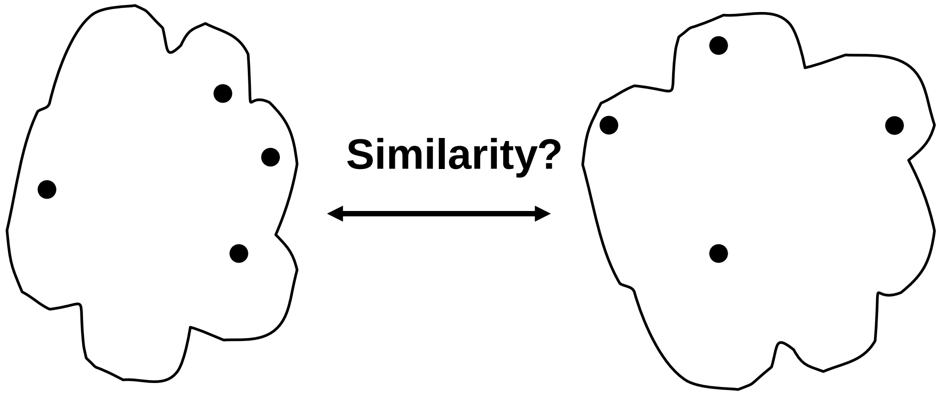


	C1	C2 U C5	C3	C4
C1		?		
C2 U C5	?	?	?	?
C3		?		
C4		?		

Proximity Matrix



How to Define Inter-Cluster Similarity

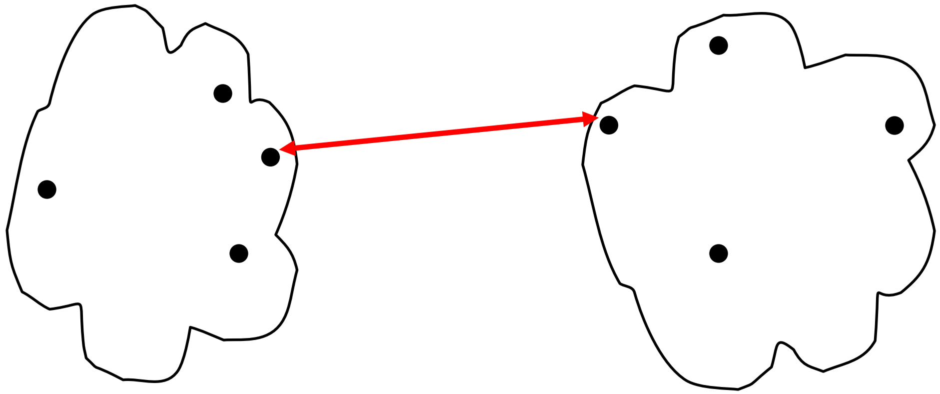


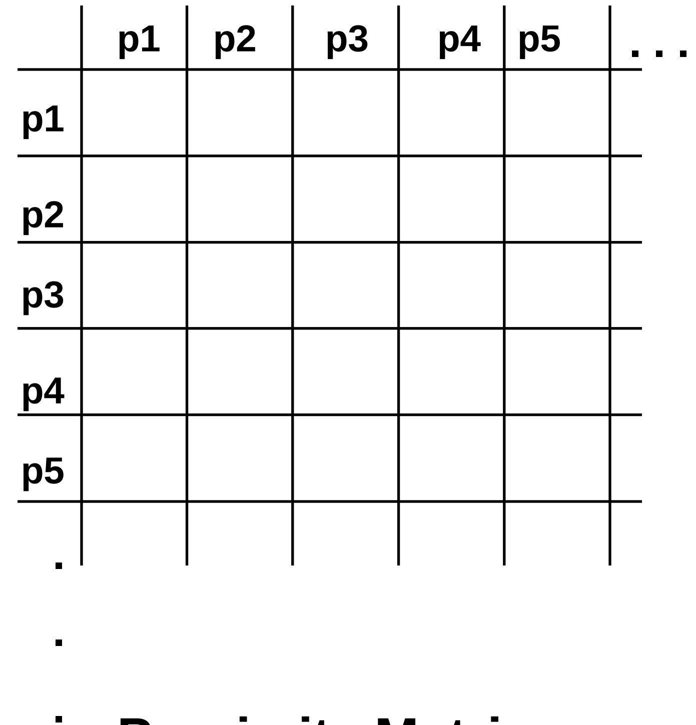
	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

How to Define Inter-Cluster Similarity

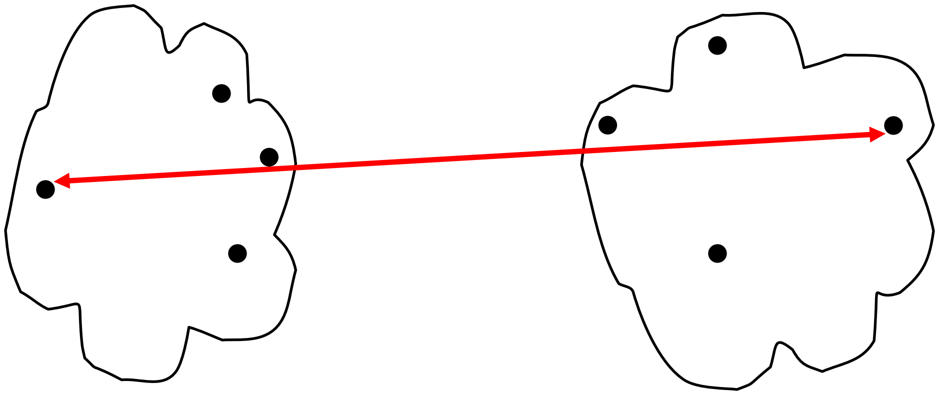




Proximity Matrix

- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

How to Define Inter-Cluster Similarity

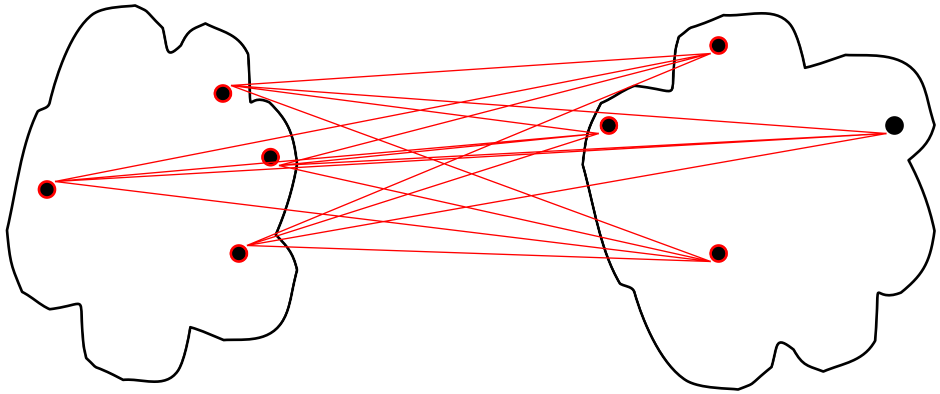


	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

How to Define Inter-Cluster Similarity

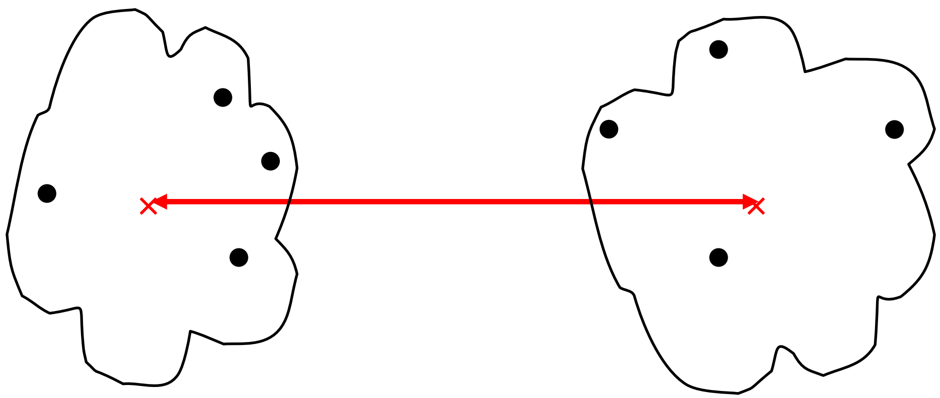


	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  - Ward's Method uses squared error

How to Define Inter-Cluster Similarity



	p1	p2	p3	p4	p5	. . .
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
  – Ward's Method uses squared error

Data

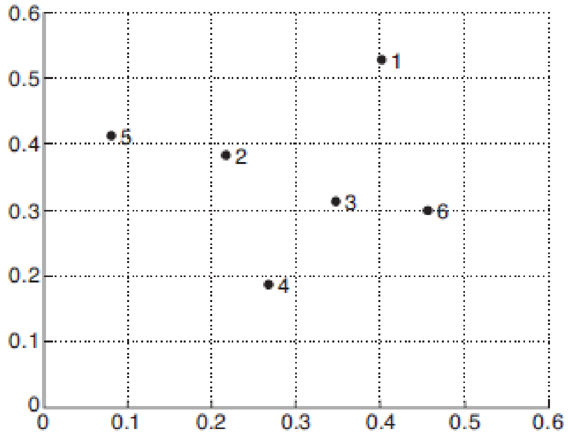


Figure 8.15. Set of 6 two-dimensional points.

Point	x Coordinate	y Coordinate
p1	0.40	0.53
p2	0.22	0.38
p3	0.35	0.32
p4	0.26	0.19
p5	0.08	0.41
p6	0.45	0.30

Table 8.3. xy coordinates of 6 points.

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

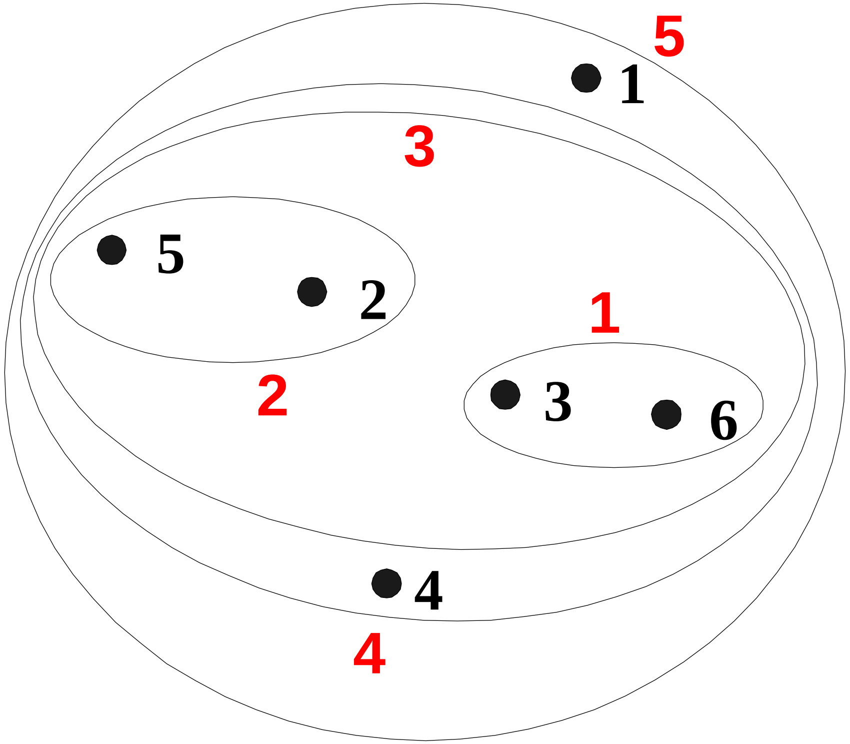
Table 8.4. Euclidean distance matrix for 6 points.

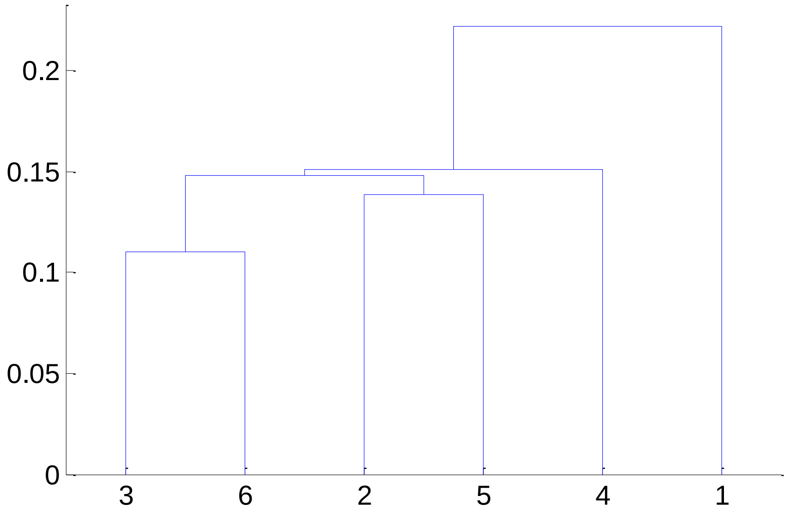
Cluster Similarity: MIN or Single Link

- Similarity of two clusters is based on the two most similar (MIN distance) points in the different clusters

- Determined by one pair of points, i.e., by one link in the proximity graph.

Hierarchical Clustering: MIN





Nested Clusters

Dendrogram

Cluster Similarity: MIN or Single Link

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

	P1	P2	P3 & P6	P4	P5
P1	0.00	0.24		0.37	0.34
P2	0.24	0.00		0.20	0.14
P3 & P6			0.00		
P4	0.37	0.20		0.00	0.29
P5	0.34	0.14		0.29	0.00

Cluster Similarity: MIN or Single Link

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

	P1	P2	P3 & P6	P4	P5
P1	0.00	0.24		0.37	0.34
P2	0.24	0.00		0.20	0.14
P3 & P6	0.22		0.00		
P4	0.37	0.20		0.00	0.29
P5	0.34	0.14		0.29	0.00

Cluster Similarity: MIN or Single Link

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

	P1	P2	P3 & P6	P4	P5
P1	0.00	0.24		0.37	0.34
P2	0.24	0.00		0.20	0.14
P3 & P6	0.22	0.15	0.00		
P4	0.37	0.20		0.00	0.29
P5	0.34	0.14		0.29	0.00

Cluster Similarity: MIN or Single Link

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

	P1	P2	P3 & P6	P4	P5
P1	0.00	0.24		0.37	0.34
P2	0.24	0.00		0.20	0.14
P3 & P6	0.22	0.15	0.00	0.15	
P4	0.37	0.20		0.00	0.29
P5	0.34	0.14		0.29	0.00

Cluster Similarity: MIN or Single Link

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

	P1	P2	P3 & P6	P4	P5
P1	0.00	0.24		0.37	0.34
P2	0.24	0.00		0.20	0.14
P3 & P6	0.22	0.15	0.00	0.15	0.28
P4	0.37	0.20		0.00	0.29
P5	0.34	0.14		0.29	0.00

Cluster Similarity: MIN or Single Link

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

	P1	P2	P3 & P6	P4	P5
P1	0.00	0.24	0.22	0.37	0.34
P2	0.24	0.00	0.15	0.20	0.14
P3 & P6	0.22	0.15	0.00	0.15	0.28
P4	0.37	0.20	0.15	0.00	0.29
P5	0.34	0.14	0.28	0.29	0.00

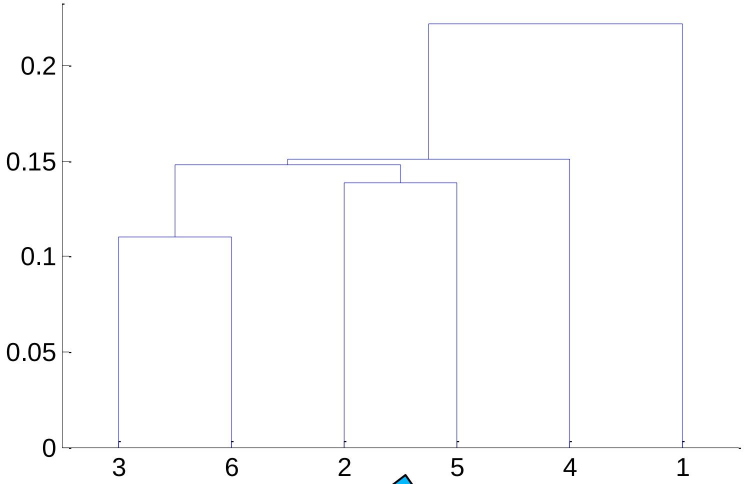
Cluster Similarity: MIN or Single Link

	P1	P2	P3 & P6	P4	P5
P1	0.00	0.24	0.22	0.37	0.34
P2	0.24	0.00	0.15	0.20	0.14
P3 & P6	0.22	0.15	0.00	0.15	0.28
P4	0.37	0.20	0.15	0.00	0.29
P5	0.34	0.14	0.28	0.29	0.00

	P1	P2 & P5	P3 & P6	P4
P1	0.00		0.22	0.37
P2 & P5		0.00		
P3 & P6	0.22		0.00	0.15
P4	0.37		0.15	0.00

Cluster Similarity: MIN or Single Link

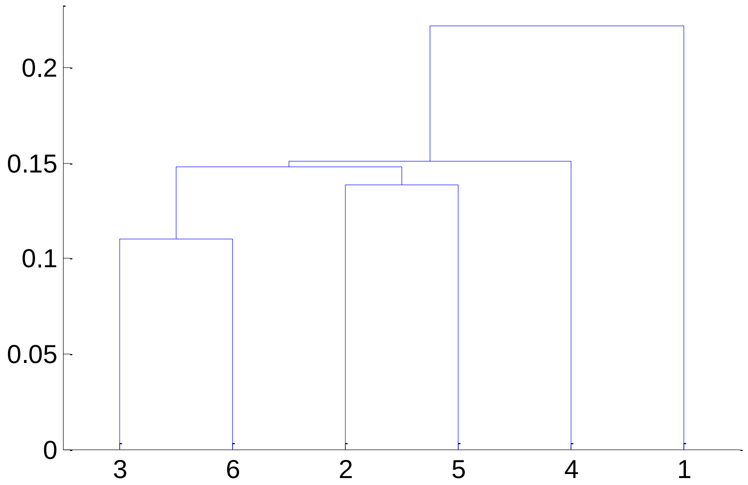
	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00



Cluster Similarity: MIN or Single Link

Distance Matrix

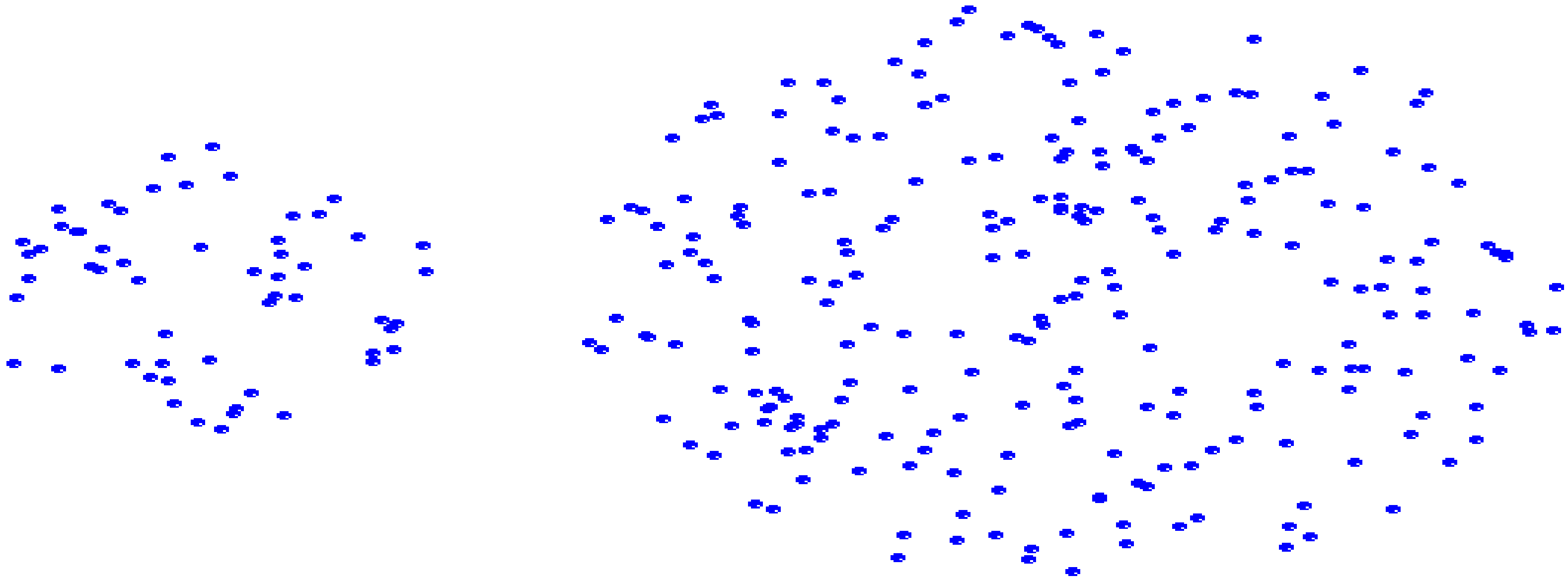
	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00



Similarity matrix

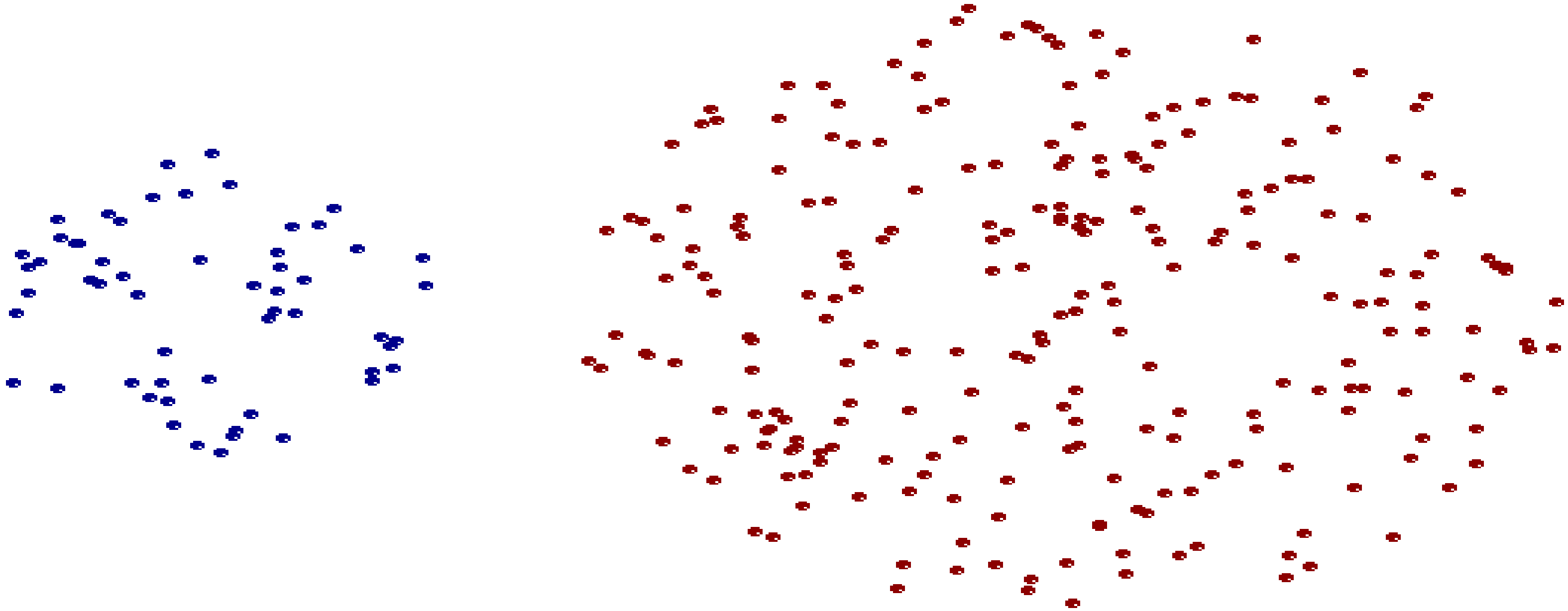
	P1	**P2**	**P3**	**P4**	**P5**	**P6**
P1	1.00	0.81	0.82	0.73	0.75	0.81
P2	0.81	1.00	0.87	0.83	0.88	0.8
P3	0.82	0.87	1.00	0.87	0.78	0.9
P4	0.73	0.83	0.87	1.00	0.77	0.81
P5	0.75	0.88	0.78	0.77	1.00	0.72
P6	0.81	0.8	0.9	0.81	0.72	1.00

Strength of MIN



Original Points

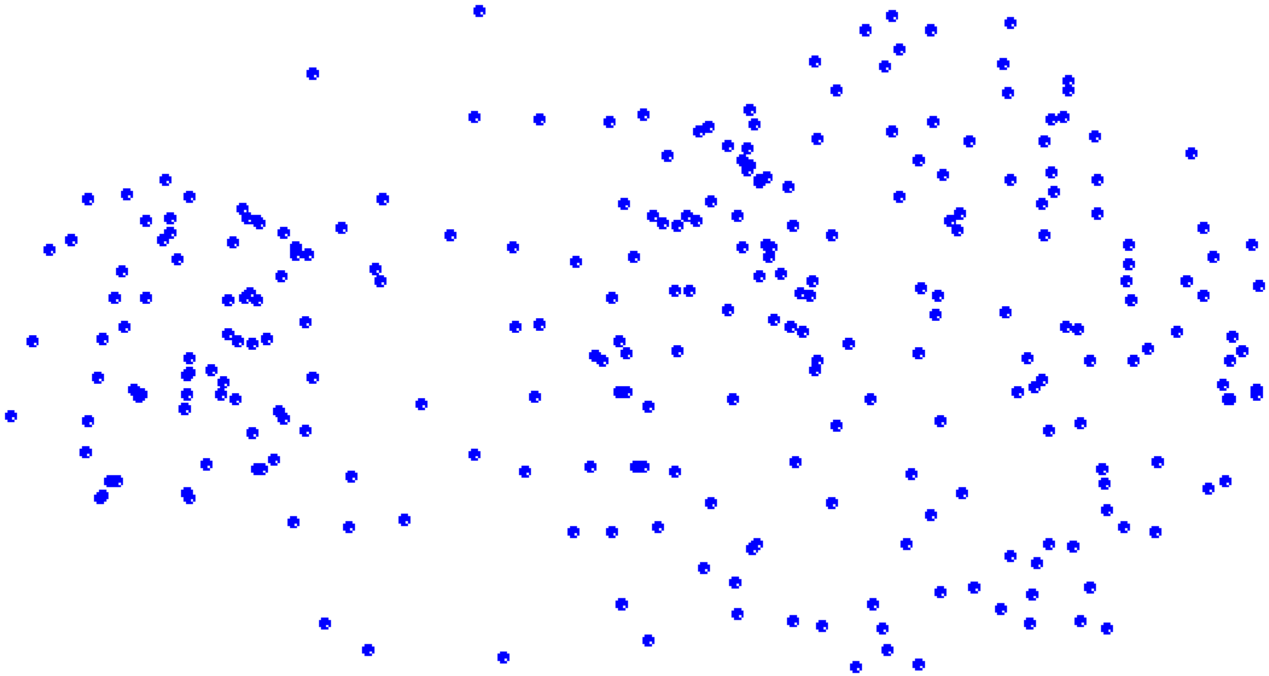
Strength of MIN



Two Clusters

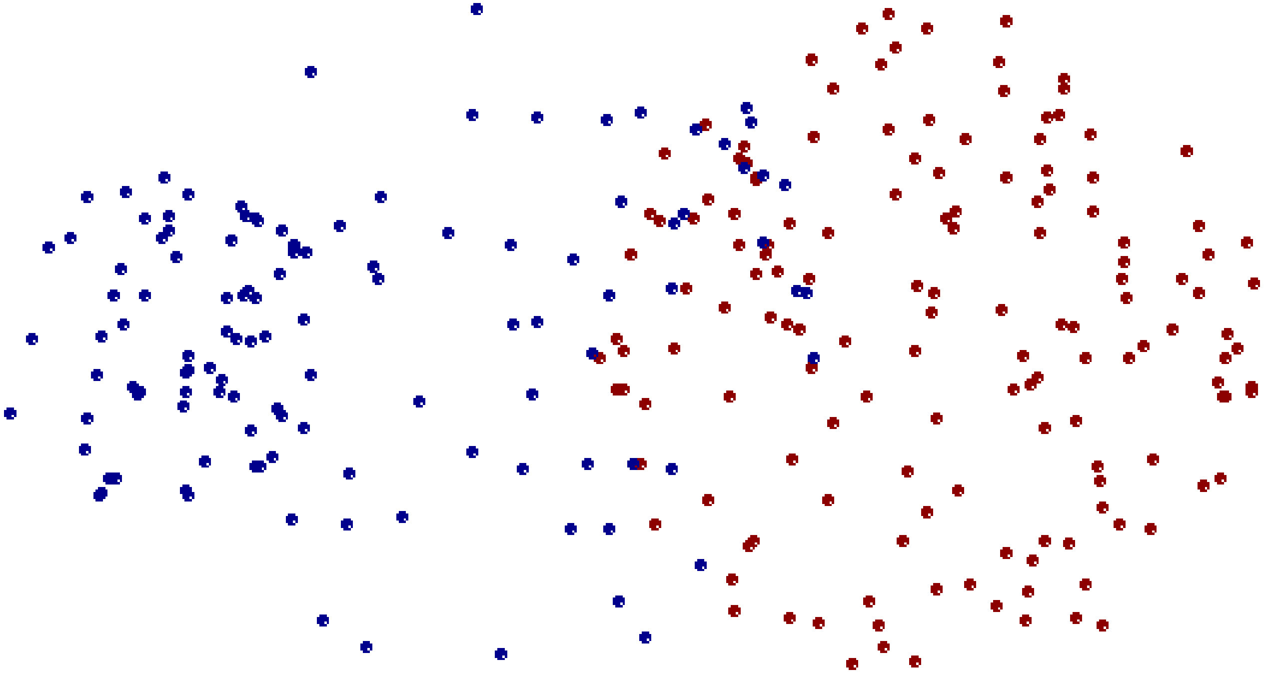
Can handle non-elliptical shapes

Limitations of MIN



Original Points

Limitations of MIN



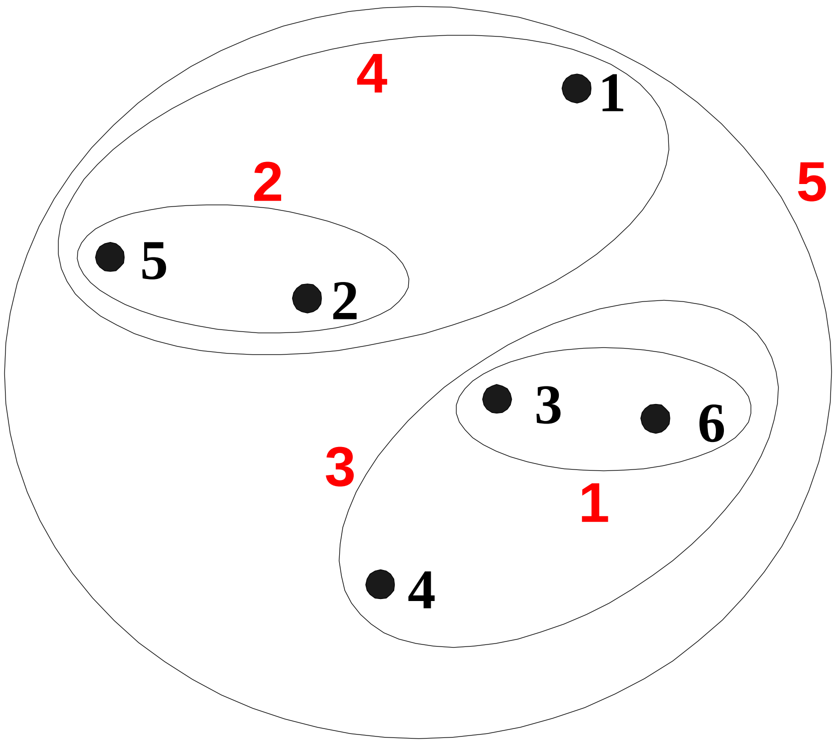
Two Clusters

Sensitive to noise and outliers

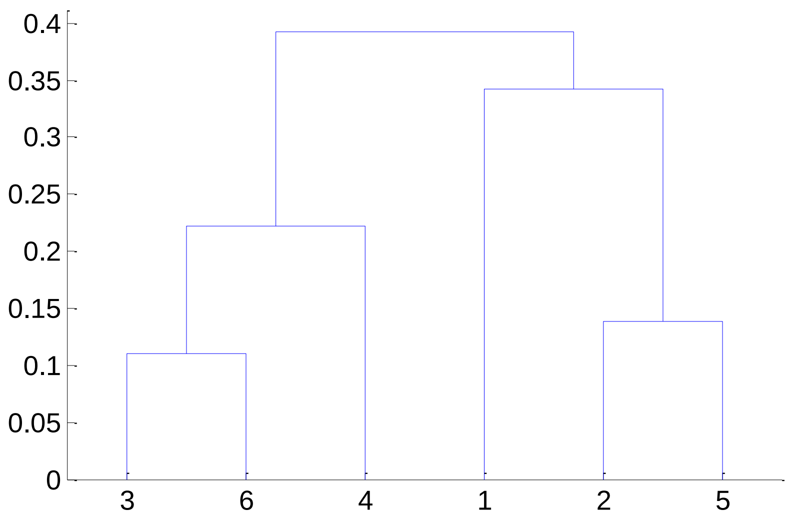
Cluster Similarity: MAX or Complete Linkage

- Similarity of two clusters is based on the two least similar (most distant) points in the different clusters
- Determined by all pairs of points in the two clusters

Hierarchical Clustering: MAX

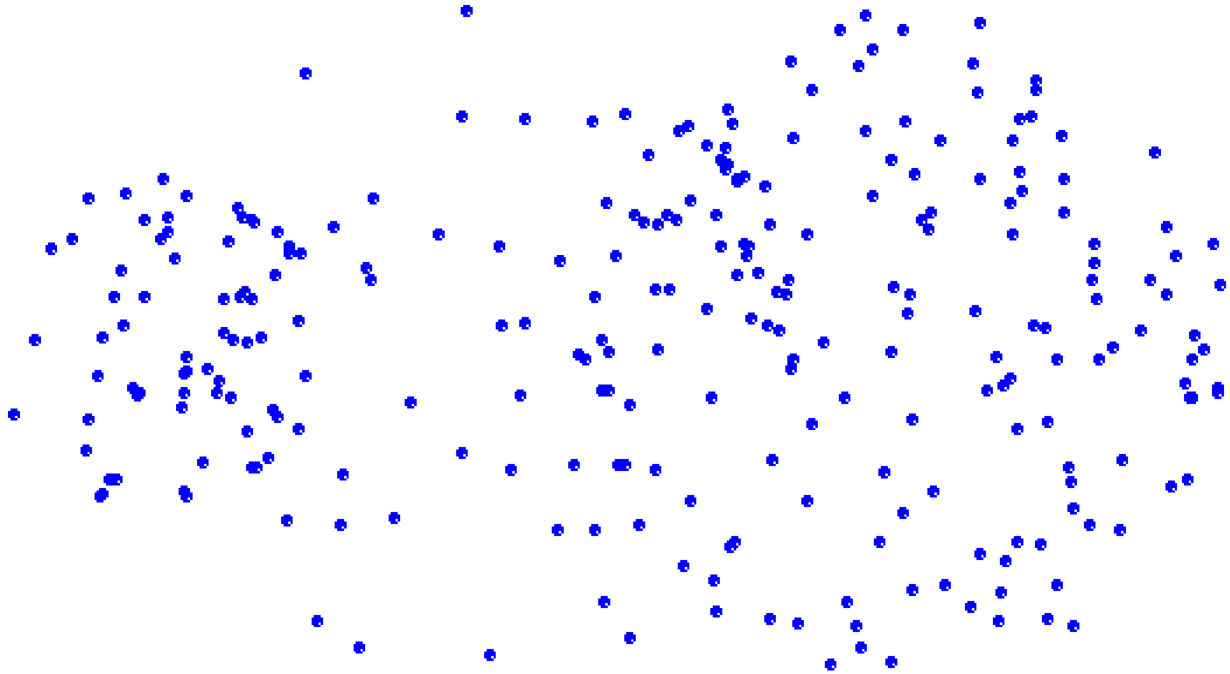


Nested Clusters



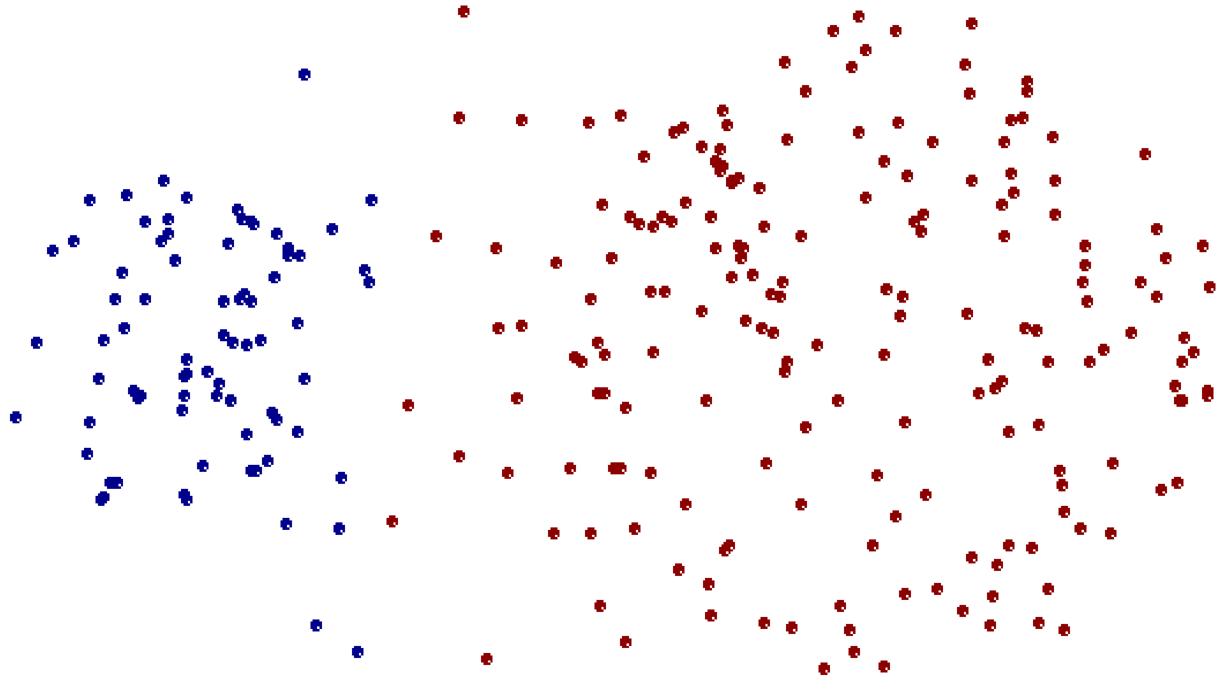
Dendrogram

Strength of MAX



Original Points

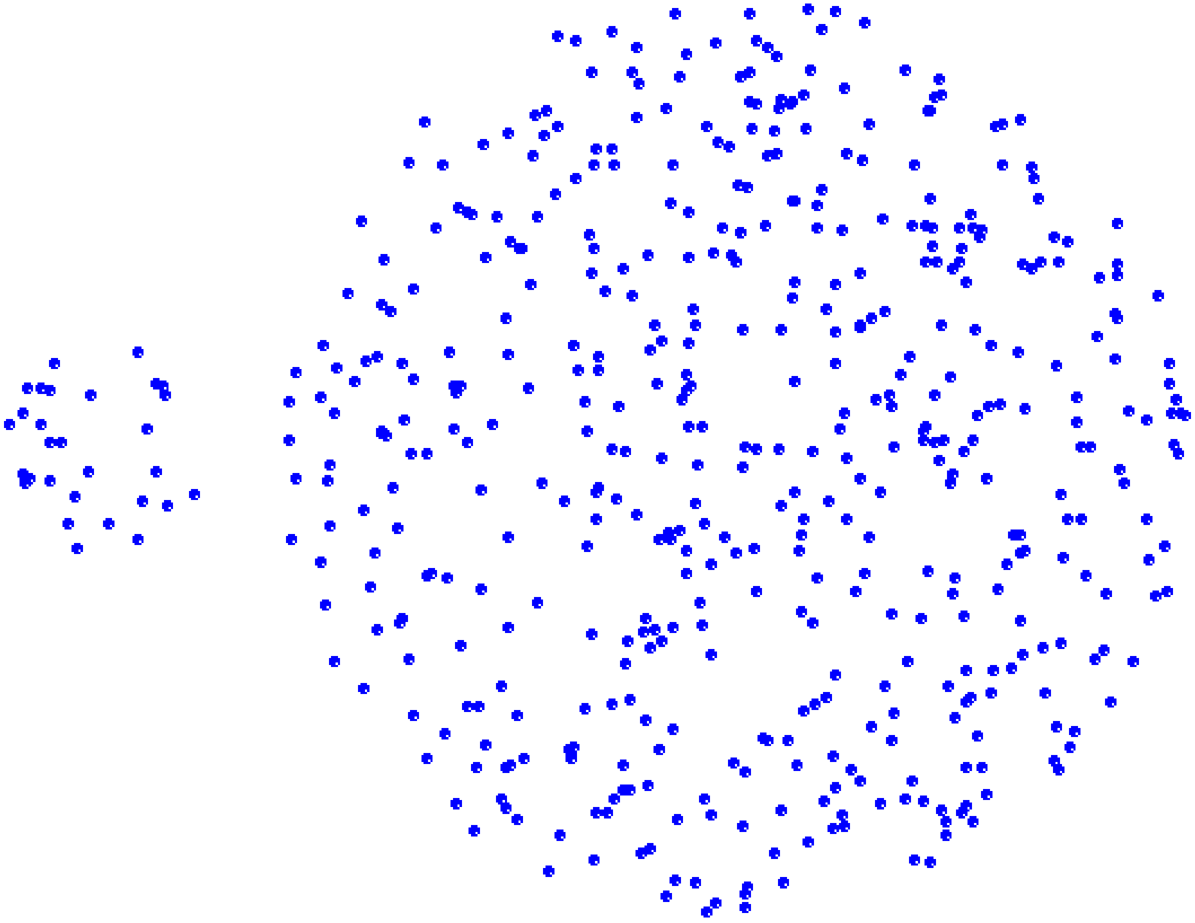
Strength of MAX



Two Clusters

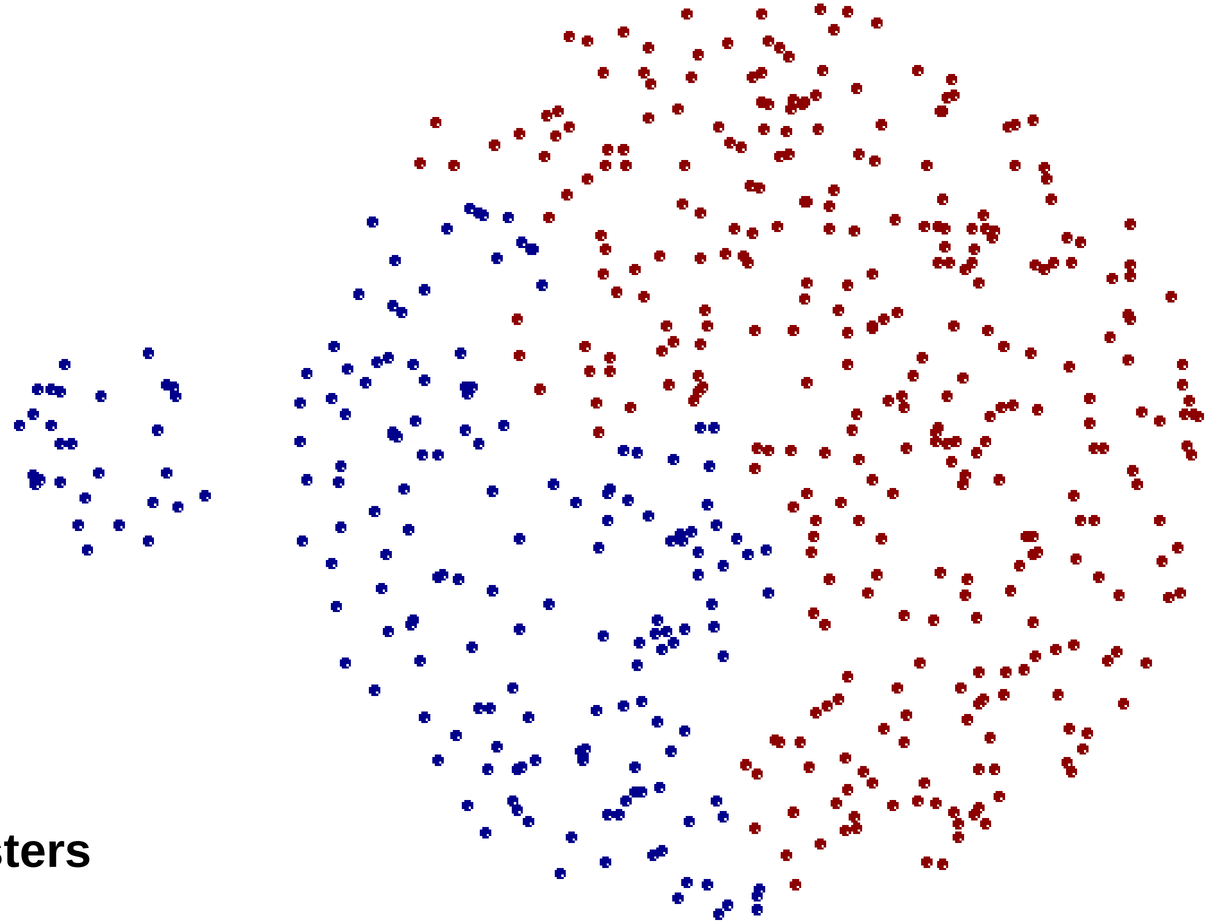
Less susceptible to noise and outliers

Limitations of MAX



Original Points

Limitations of MAX



Two Clusters

Tends to break large clusters

Biased towards globular clusters

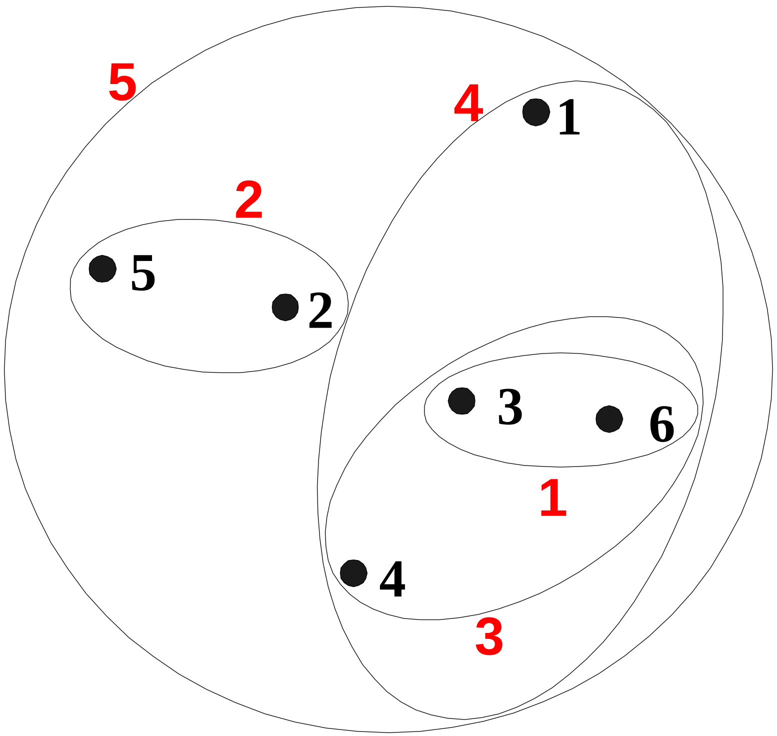
Cluster Similarity: Group Average

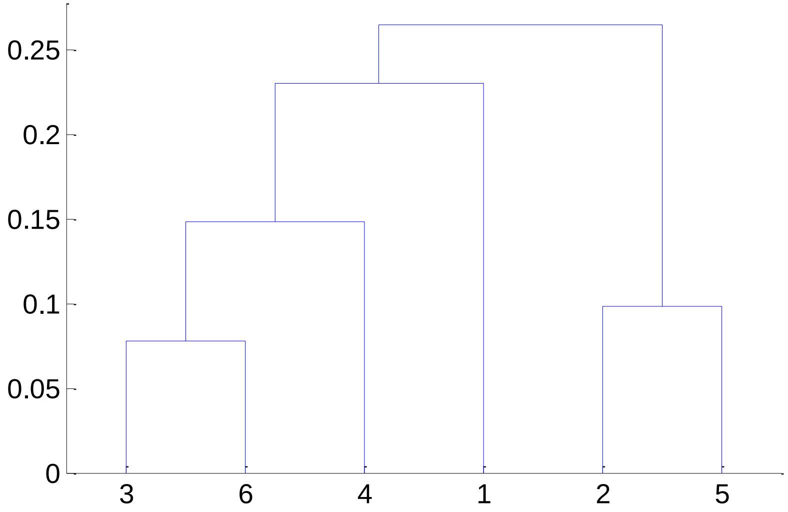
- Proximity of two clusters is the average of pairwise proximity between points in the two clusters.

$$\text{proximity}(\text{Cluster}_i, \text{Cluster}_j) = \frac{\sum\limits_{\substack{p_i \in \text{Cluster}_i \\ p_j \in \text{Cluster}_j}} \text{proximity}(p_i, p_j)}{|\text{Cluster}_i| * |\text{Cluster}_j|}$$

- Need to use average connectivity for scalability since total proximity favors large clusters

Hierarchical Clustering: Group Average





Nested Clusters

Dendrogram

Hierarchical Clustering: Group Average

- Compromise between Single and Complete Link

- Strengths

  Less susceptible to noise and outliers

- Limitations

  Biased towards globular clusters

Distance between two clusters

Ward's distance between clusters C_i and C_j is the ***difference*** between the ***total within cluster sum of squares for the two clusters separately***, and the ***within cluster sum of squares resulting from merging the two clusters*** in cluster C_{ij}

$$D_w(C_i, C_j) = \sum_{x \in C_i} (x - m_i)^2 + \sum_{x \in C_j} (x - m_j)^2 - \sum_{x \in C_{ij}} (x - m_{ij})^2$$

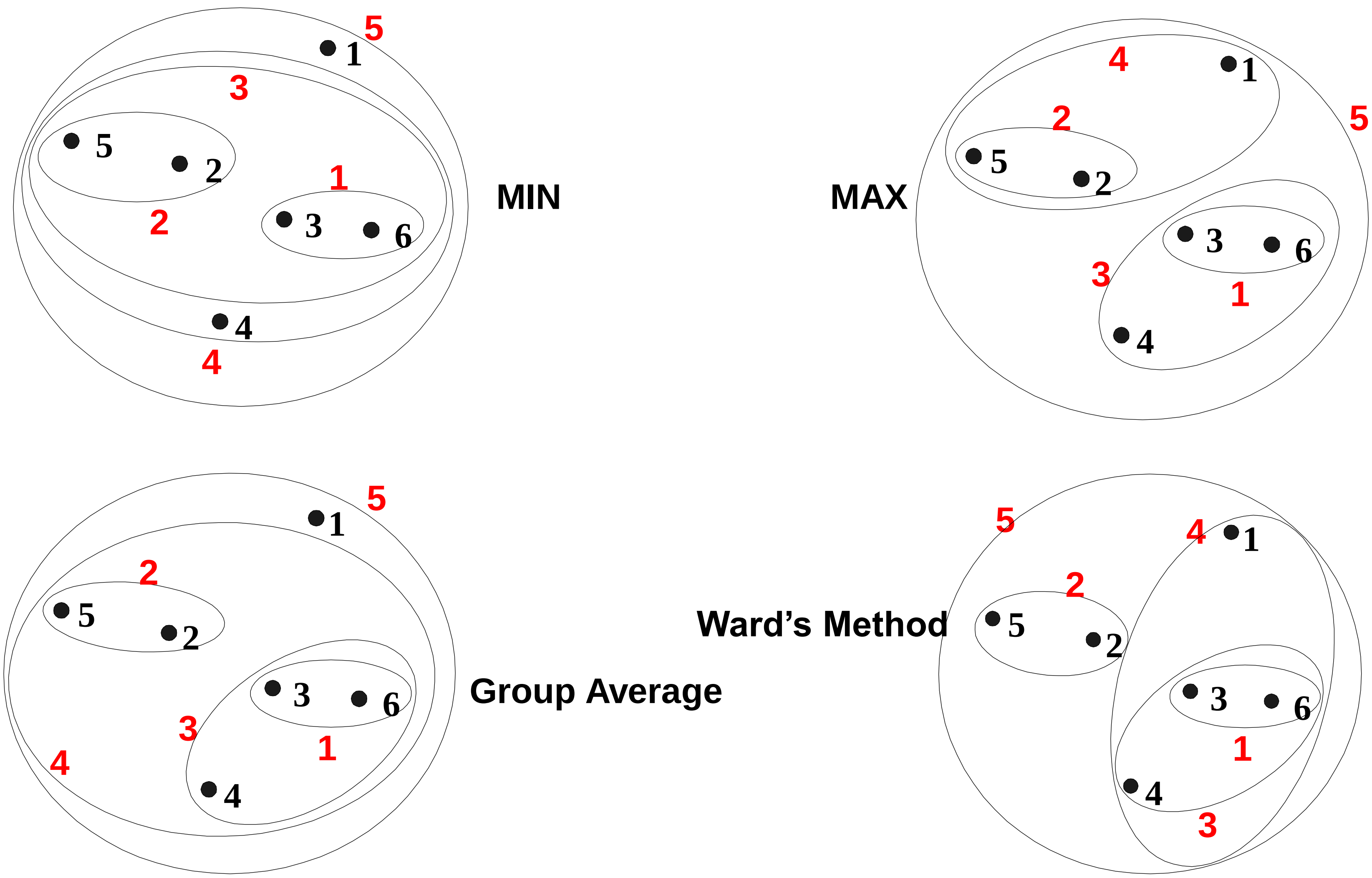
m_i: centroid of C_i
m_j: centroid of C_j
m_{ij}: centroid of C_{ij}

Cluster Similarity: Ward's Method

- *Similarity of two clusters is based on the increase in squared error when two clusters are merged (Ward's distance)*
- *Similar* to **group average** if distance between points is distance squared
- *Less susceptible* to **noise** and **outliers**
- *Biased* towards **globular clusters**
- *Hierarchical analogue* of **K-means**

  Can be used to initialize K-means

Hierarchical Clustering: Comparison



Hierarchical Clustering: Time and Space requirements

- $O(N^2)$ space since it uses the proximity matrix.

  N is the number of points.

- $O(N^3)$ time in many cases

  - There are N steps and at each step the size, N^2, proximity matrix must be updated and searched

  - Complexity can be reduced to $O(N^2 \log(N))$ time for some approaches

Hierarchical Clustering: Problems and Limitations

- Once a decision is made to combine two clusters, it cannot be undone

- No objective function is directly minimized

- Different schemes have problems with one or more of the following:

  - Sensitivity to noise and outliers
  - Difficulty handling different sized clusters and convex shapes
  - Breaking large clusters

scipy.cluster.hierarchy.linkage

scipy.cluster.hierarchy.linkage(y, method='single', metric='euclidean', optimal_ordering=False)

Perform hierarchical/agglomerative clustering

- **method** str, optional: { 'single','complete','average'}
  The linkage algorithm to use.
- **metric** str or function, optional
  The distance metric to use
- **optimal_ordering** bool, optional
  If True, the linkage matrix will be reordered so that the distance between successive leaves is minimal. This results in a more intuitive tree structure when the data are visualized.

sklearn.cluster.AgglomerativeClustering

*class sklearn.cluster.AgglomerativeClustering(n_clusters=2, *, affinity='euclidean', memory=None, connectivity=None, compute_full_tree='auto', linkage='ward', distance_threshold=None, compute_distances=False)*

Recursively merges the pair of clusters that minimally increases a given linkage distance.

- **n_clusters** int or None, default=2 The number of clusters to find.
- **affinity** str or callable, default='euclidean'
  Metric used to compute the linkage.
- **linkage** {'ward', 'complete', 'average', 'single'}, default='ward'
  Which linkage criterion to use.
- **distance_threshold** float, default=None
  The linkage distance threshold above which, clusters will not be merged.

Cluster Validity

Cluster Validity

- For supervised classification we have a variety of measures to evaluate how good our model is
  - Accuracy, precision, recall

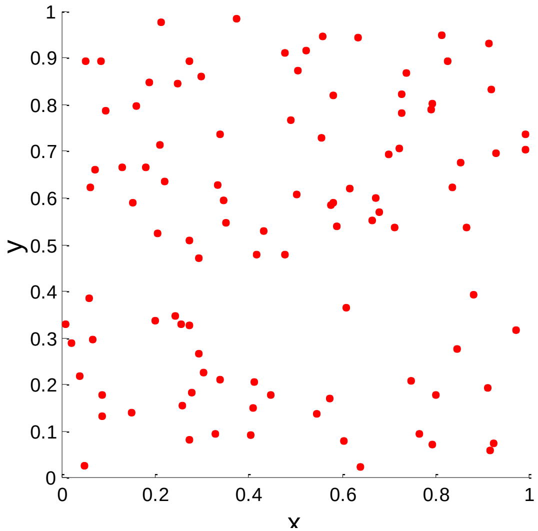
- For cluster analysis, the analogous question is how to evaluate the "goodness" of the resulting clusters?

- But "clusters are in the eye of the beholder"!

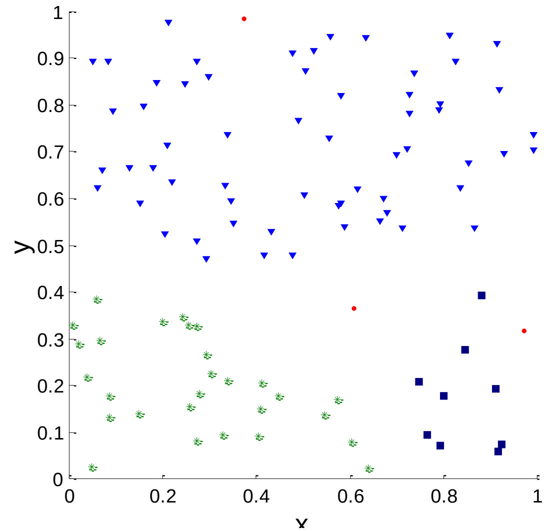
- Then why do we want to evaluate them?
  - To avoid finding patterns in noise
  - To compare clustering algorithms
  - To compare two sets of clusters
  - To compare two clusters

Clusters found in Random Data

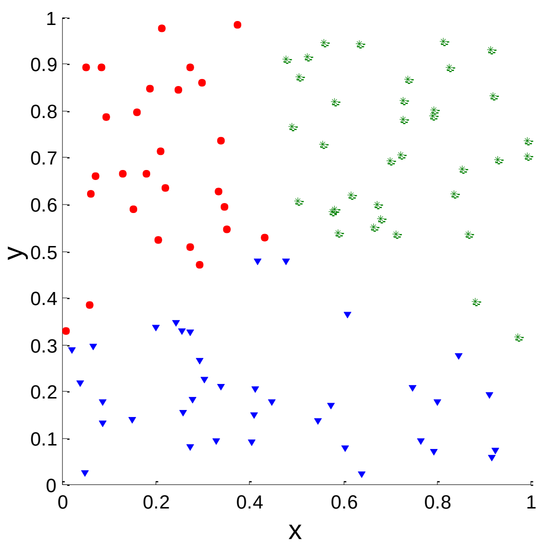
Random Points



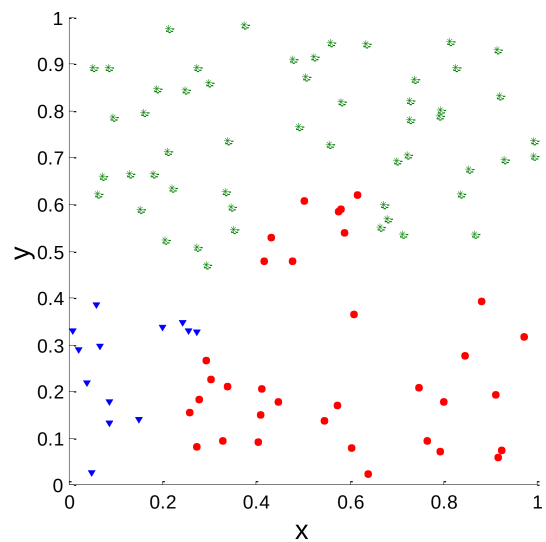
DBSCAN



K-means



Complete Link



Different Aspects of Cluster Validation

1. Determining the clustering tendency of a set of data, i.e., distinguishing whether non-random structure actually exists in the data.
2. Comparing the results of a cluster analysis to externally known results, e.g., to externally given class labels.
3. Evaluating how well the results of a cluster analysis fit the data *without* reference to external information.

   - Use only the data

4. Comparing the results of two different sets of cluster analyses to determine which is better.
5. Determining the 'correct' number of clusters.

For 2, 3, and 4, we can further distinguish whether we want to evaluate the entire clustering or just individual clusters.

Formulating the Problem

- Most important is selecting the variables on which the clustering is based.
- Inclusion of even one or two irrelevant variables may distort a clustering solution.
- Variables selected should describe the similarity between objects in terms that are relevant to the marketing research problem.
- Should be selected based on past research, theory, or a consideration of the hypotheses being tested.

Measures of Cluster Validity

- Numerical measures that are applied to judge various aspects of cluster validity, are classified into the following three types.

  - External Index: Used to measure the extent to which cluster labels match externally supplied class labels.

    Entropy

  - Internal Index: Used to measure the goodness of a clustering structure *without* respect to external information.

    Sum of Squared Error (SSE)

  - Relative Index: Used to compare two different clusterings or clusters.

    Often an external or internal index is used for this function, e.g., SSE or entropy

- Sometimes these are referred to as criteria instead of indices

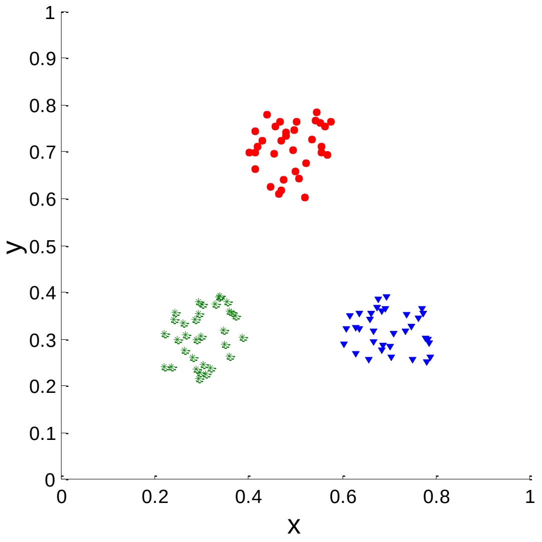
  However, sometimes criterion is the general strategy and index is the numerical measure that implements the criterion.

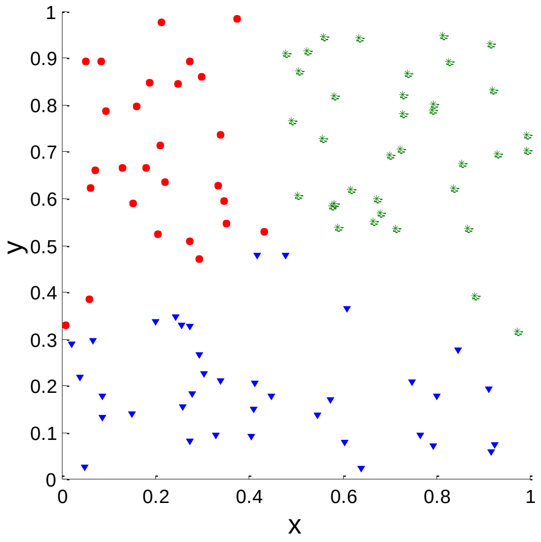
Measuring Cluster Validity Via Correlation

- Two matrices
  - Proximity Matrix
  - "Incidence" Matrix
    - One row and one column for each data point
    - An entry is 1 if the associated pair of points belong to the same cluster
    - An entry is 0 if the associated pair of points belongs to different clusters
- Compute the correlation between the two matrices
  - Since the matrices are symmetric, only the correlation between $n(n-1) / 2$ entries needs to be calculated.
- High correlation indicates that points that belong to the same cluster are close to each other.
- Not a good measure for some density or contiguity based clusters.

Measuring Cluster Validity Via Correlation

- Correlation of incidence and proximity matrices for the K-means clusterings of the following two data sets.



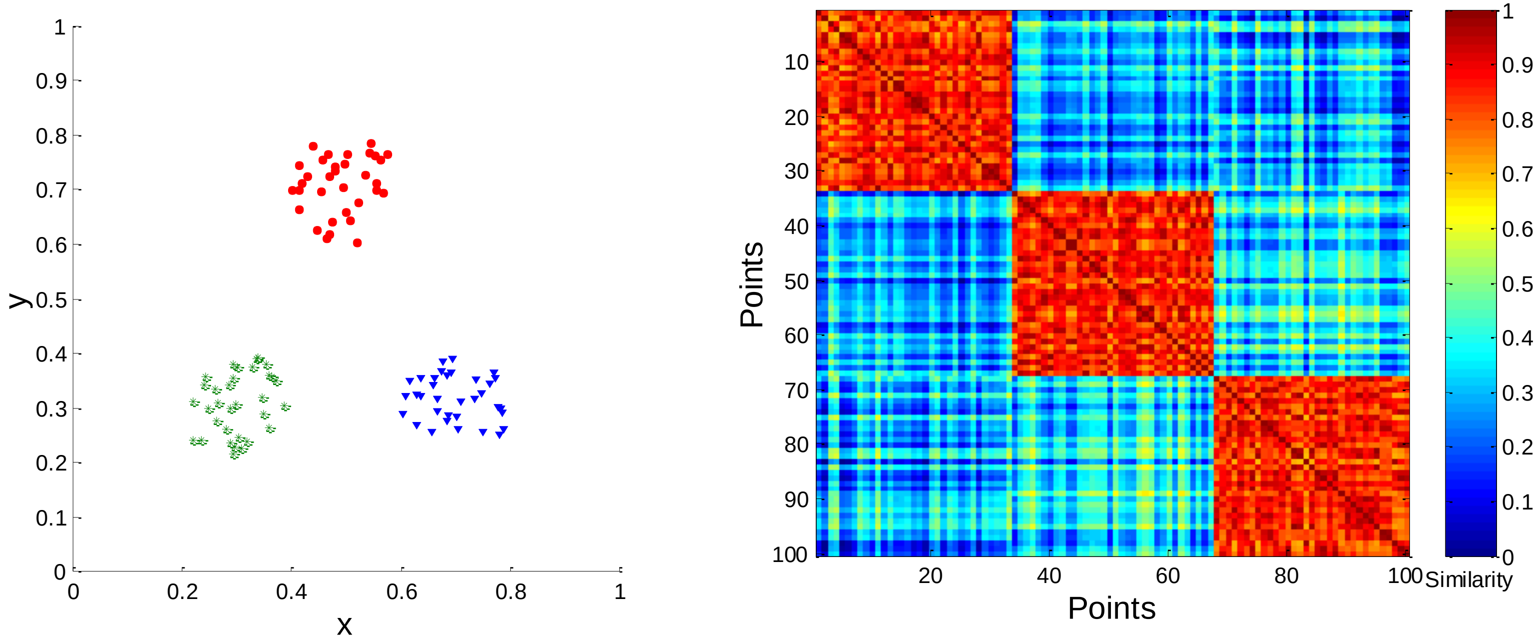


Corr = -0.9235

Corr = -0.5810

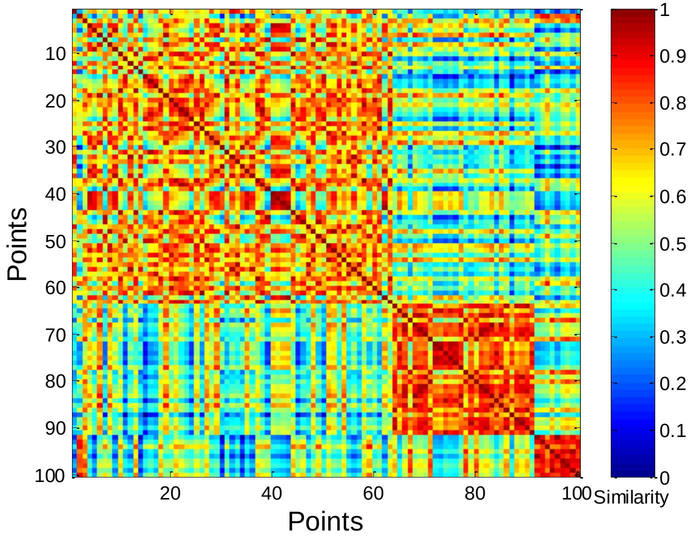
Using Similarity Matrix for Cluster Validation

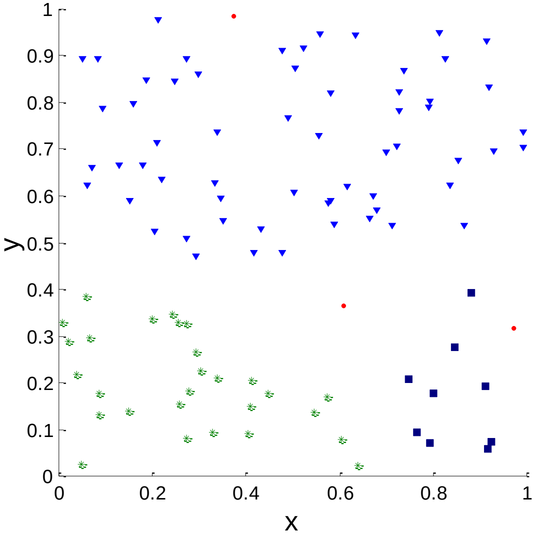
- Order the similarity matrix with respect to cluster labels and inspect visually.



Using Similarity Matrix for Cluster Validation

Clusters in random data are not so crisp

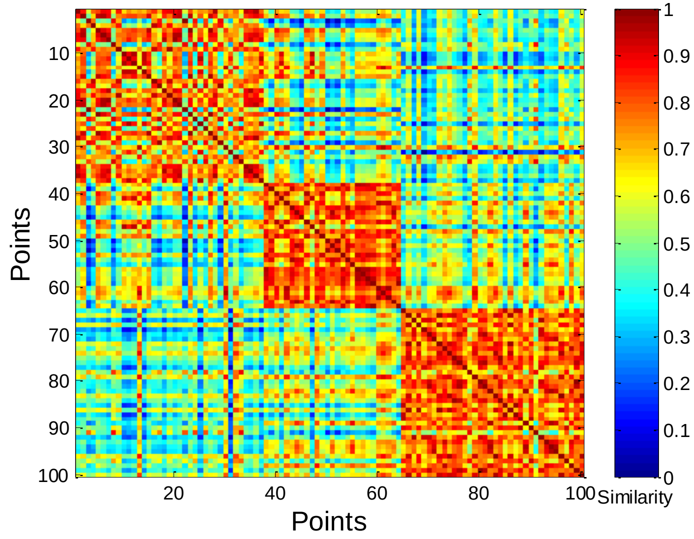


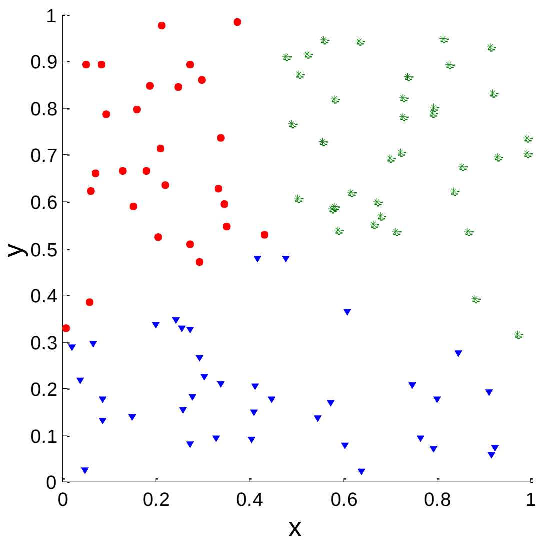


DBSCAN

Using Similarity Matrix for Cluster Validation

- Clusters in random data are not so crisp

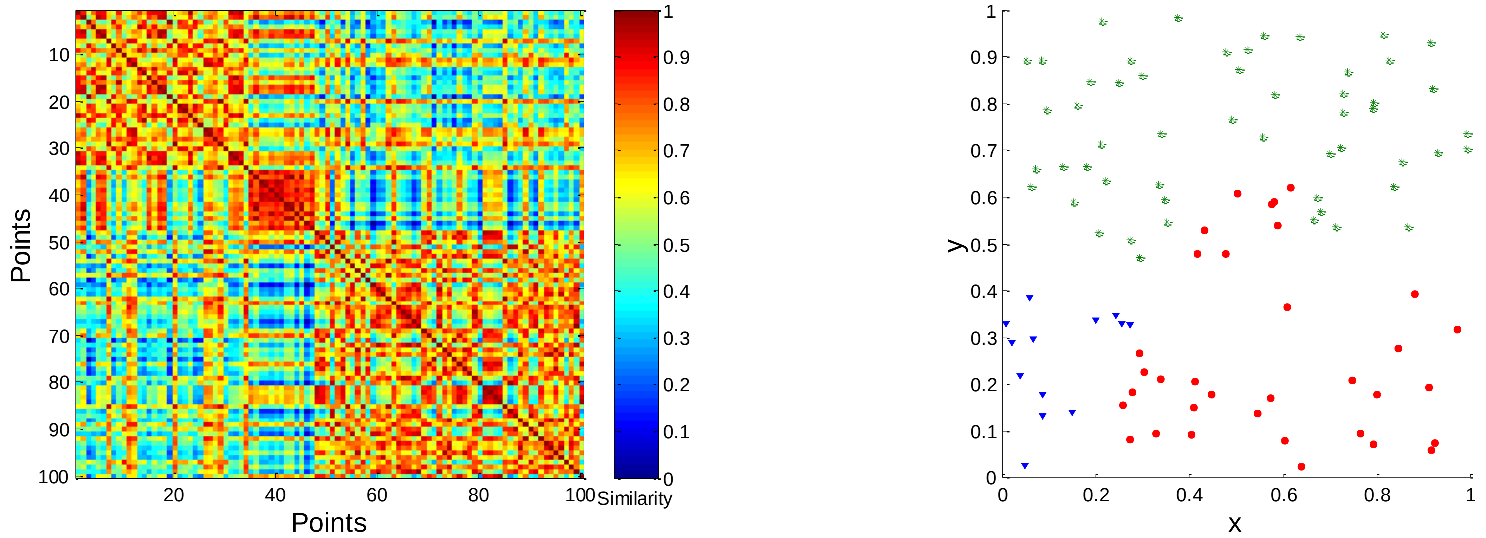




K-means

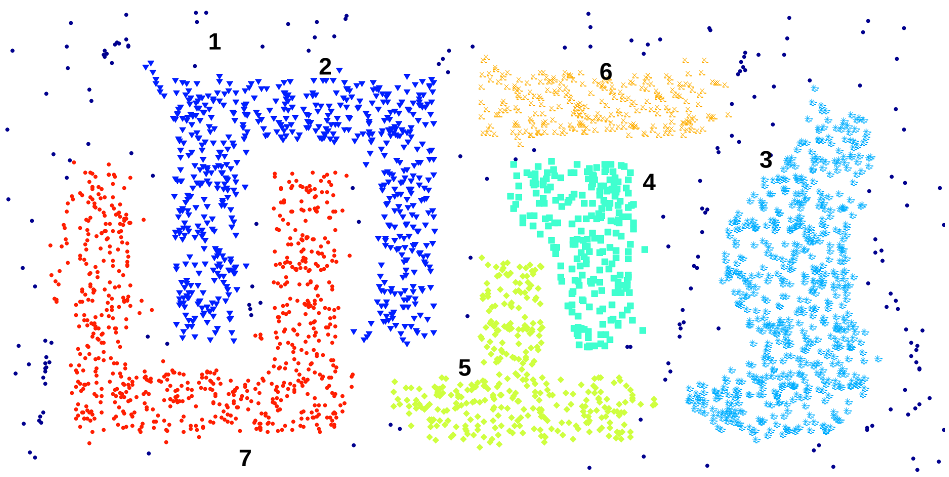
Using Similarity Matrix for Cluster Validation

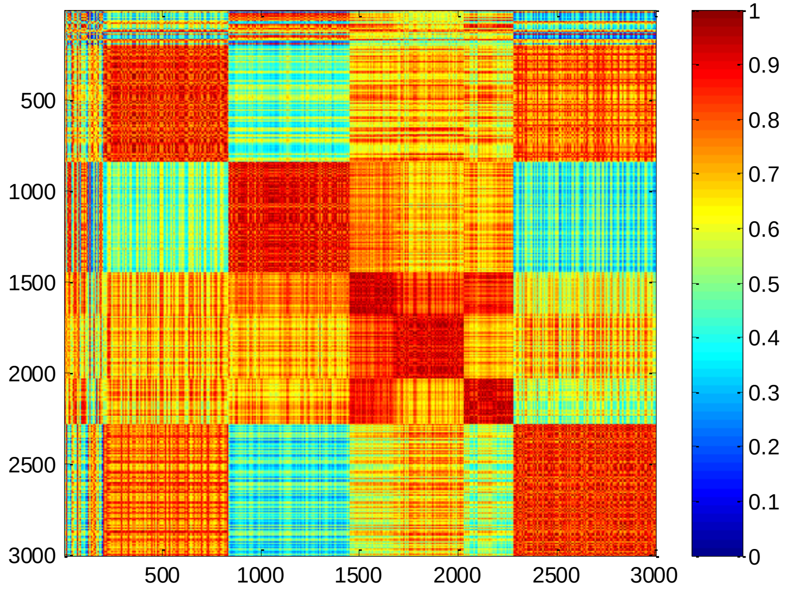
- Clusters in random data are not so crisp



Complete Link

Using Similarity Matrix for Cluster Validation





DBSCAN

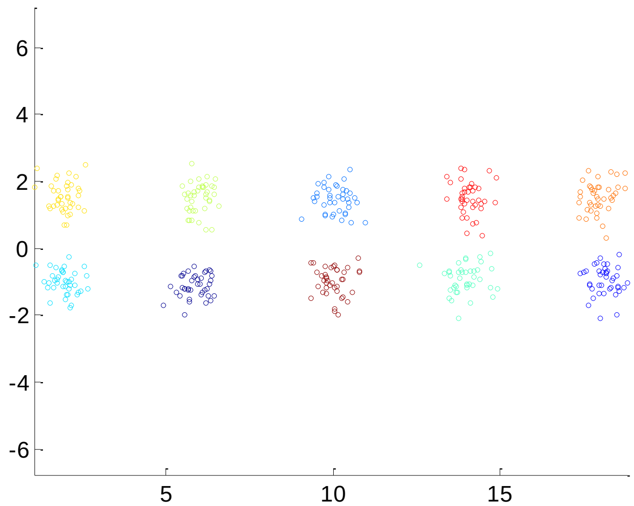
Internal Measures: SSE

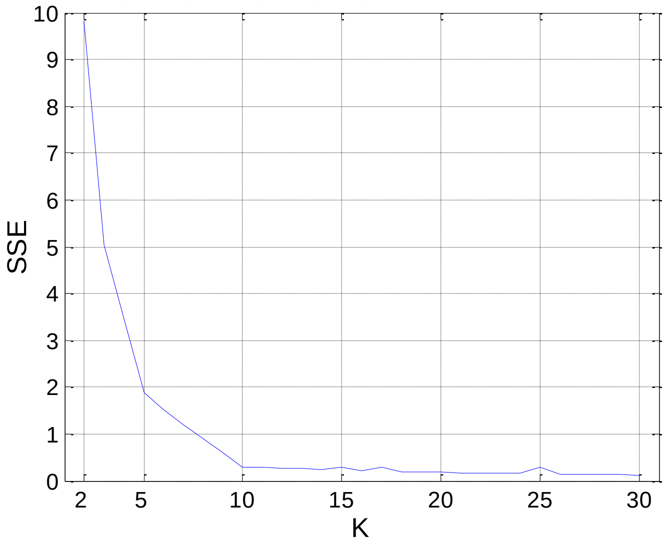
- Clusters in more complicated figures aren't well separated

Internal Index: Used to measure the goodness of a clustering structure without respect to external information SSE

SSE is good for comparing two clusterings or two clusters (average SSE).

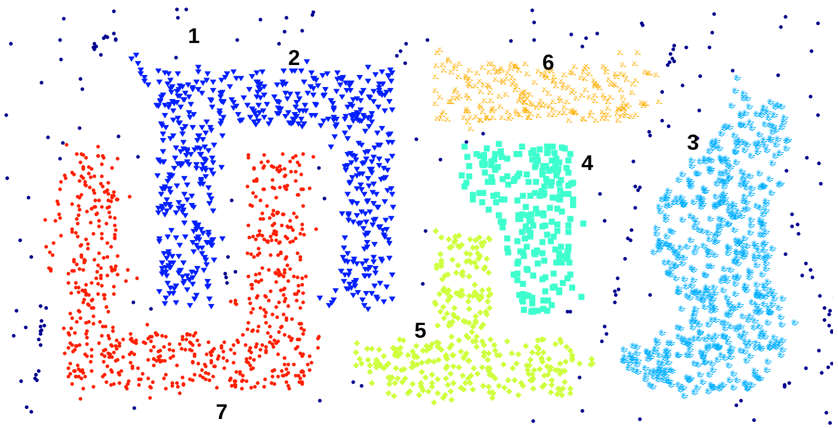
Can also be used to estimate the number of clusters

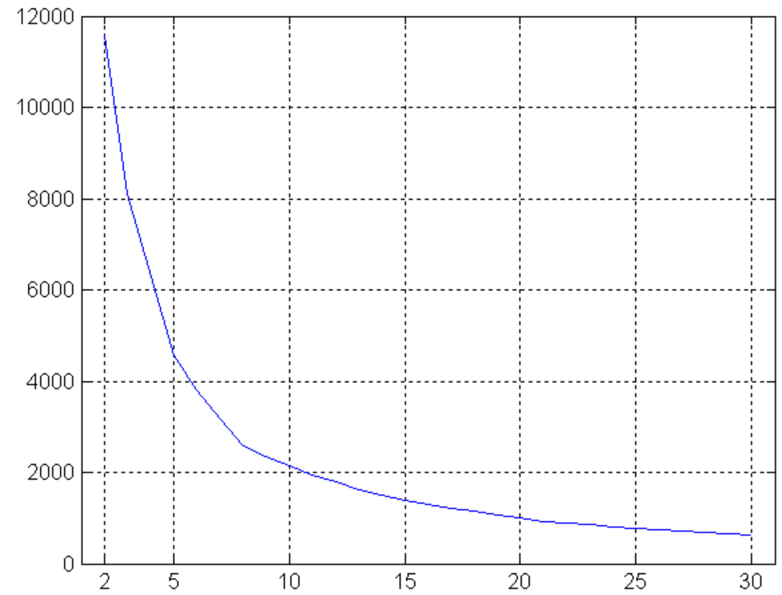




Internal Measures: SSE

SSE curve for a more complicated data set



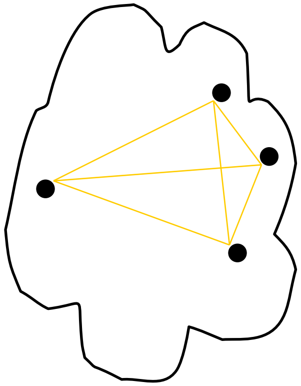


SSE of clusters found using K-means

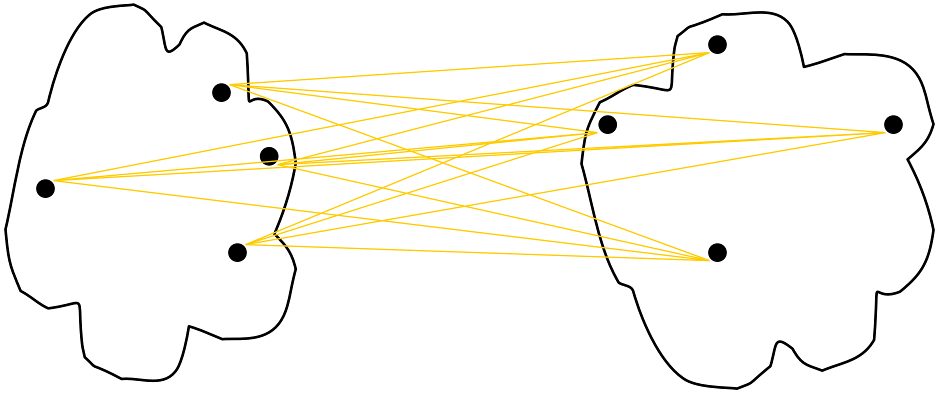
Internal Measures: Cohesion and Separation

A proximity graph based approach can also be used for cohesion and separation.

- Cluster cohesion is the sum of the weight of all links within a cluster.
- Cluster separation is the sum of the weights between nodes in the cluster and nodes outside the cluster.



cohesion



separation

Internal Measures: Cohesion and Separation

Cluster Cohesion: Measures how closely related are objects in a cluster

Example: SSE

Cluster Separation: Measure how distinct or well-separated a cluster is from other clusters

Example: Squared Error

Cohesion is measured by the within cluster sum of squares (SSE)

$$WSS = \sum_{i} \sum_{x \in C_i} (x - m_i)^2$$

Separation is measured by the between cluster sum of squares

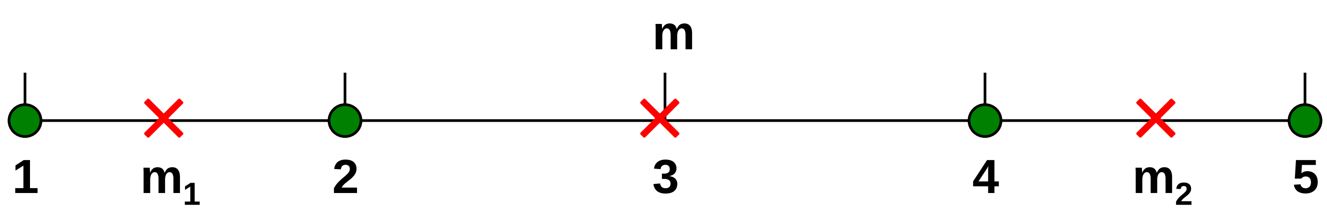
$$BSS = \sum_{i} |C_i| (m - m_i)^2$$

Where $|C_i|$ is the size of cluster *i*

Internal Measures: Cohesion and Separation

Example: SSE

BSS + WSS = constant



K=1 cluster:

$$WSS = (1-3)^2 + (2-3)^2 + (4-3)^2 + (5-3)^2 = 10$$

$$BSS = 4 \times (3-3)^2 = 0$$

$$Total = 10 + 0 = 10$$

K=2 clusters:

$$WSS = (1-1.5)^2 + (2-1.5)^2 + (4-4.5)^2 + (5-4.5)^2 = 1$$

$$BSS = 2 \times (3-1.5)^2 + 2 \times (4.5-3)^2 = 9$$

$$Total = 1 + 9 = 10$$

Internal Measures: Silhouette Coefficient

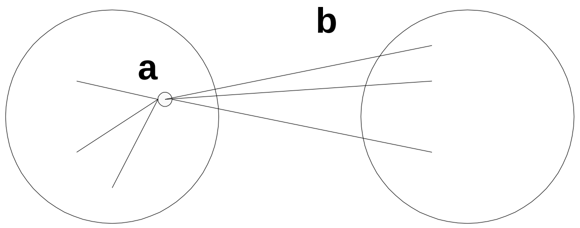
Silhouette Coefficient combine ideas of both cohesion and separation, but for individual points, as well as clusters and clusterings

For an individual point, *i*

- Calculate ***a*** = average distance of *i* to the points in its cluster
- Calculate ***b*** = min (average distance of *i* to points in another cluster)
- The silhouette coefficient for a point is then given by

$$s = 1 - a/b \quad \text{if } a < b, \quad (\text{or } s = b/a - 1 \quad \text{if } a \geq b, \text{ not the usual case})$$

- Typically between 0 and 1.
- The closer to 1 the better.



Can calculate the Average Silhouette width for a cluster or a clustering

External Measures of Cluster Validity: Entropy and Purity

Entropy: The degree to which each cluster consists of objects of a single class. For each cluster, the class distribution of the data is calculated first, i.e., for cluster j we compute p_{ij}, the probability that a member of cluster i belongs to class j as $p_{ij} = m_{ij}/m_i$, where m_i is the number of objects in cluster i and m_{ij} is the number of objects of class j in cluster i. Using this class distribution, the entropy of each cluster i is calculated using the standard formula, $e_i = -\sum_{j=1}^{L} p_{ij} \log_2 p_{ij}$, where L is the number of classes. The total entropy for a set of clusters is calculated as the sum of the entropies of each cluster weighted by the size of each cluster, i.e., $e = \sum_{i=1}^{K} \frac{m_i}{m} e_i$, where K is the number of clusters and m is the total number of data points.

External Measures of Cluster Validity: Entropy and Purity

Purity: Another measure of the extent to which a cluster contains objects of a single class. Using the previous terminology, the purity of cluster i is $p_i = \max_j p_{ij}$, the overall purity of a clustering is $purity = \sum_{i=1}^{K} \frac{m_i}{m} p_i$.

External Measures of Cluster Validity: Entropy and Purity

Table 5.9. K-means Clustering Results for LA Document Data Set

Cluster	Entertainment	Financial	Foreign	Metro	National	Sports	Entropy	Purity
1	3	5	40	506	96	27	1.2270	0.7474
2	4	7	280	29	39	2	1.1472	0.7756
3	1	1	1	7	4	671	0.1813	0.9796
4	10	162	3	119	73	2	1.7487	0.4390
5	331	22	5	70	13	23	1.3976	0.7134
6	5	358	12	212	48	13	1.5523	0.5525
Total	354	555	341	943	273	738	1.1450	0.7203

entropy For each cluster, the class distribution of the data is calculated first, i.e., for cluster j we compute p_{ij}, the 'probability' that a member of cluster j belongs to class i as follows: $p_{ij} = m_{ij}/m_j$, where m_j is the number of values in cluster j and m_{ij} is the number of values of class i in cluster j. Then using this class distribution, the entropy of each cluster j is calculated using the standard formula $e_j = \sum_{i=1}^{L} p_{ij} \log_2 p_{ij}$, where the L is the number of classes. The total entropy for a set of clusters is calculated as the sum of the entropies of each cluster weighted by the size of each cluster, i.e., $e = \sum_{i=1}^{K} \frac{m_i}{m} e_j$, where m_j is the size of cluster j, K is the number of clusters, and m is the total number of data points.

purity Using the terminology derived for entropy, the purity of cluster j, is given by $purity_j = \max p_{ij}$ and the overall purity of a clustering by $purity = \sum_{i=1}^{K} \frac{m_i}{m} purity_j$.

Final Comment on Cluster Validity

"The validation of clustering structures is the most difficult and frustrating part of cluster analysis.

Without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage."

Algorithms for Clustering Data, Jain and Dubes